

Greening the Pipeline:

An Empirical Comparison of CI/CD Refinement
Strategies and Their Carbon Impact Across Open-Source
Projects

Umer Karachiwala



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

Dún na nGall

Donegal

Department of Computing, ATU Donegal, Port Road, Letterkenny, Co. Donegal, Ireland.

Greening the Pipeline:
An Empirical Comparison of CI/CD Refinement
Strategies and Their Carbon Impact Across Open-
Source Projects

Author: Umer Karachiwala

Supervised by: Dr. Saim Ghafoor

A thesis submitted in partial fulfilment of the
requirements for the
Master of Science in Computing in DevOps

Declaration

I hereby certify that the material, which I now submit for assessment on the programmes of study leading to the award of Master of Science in Computing in **DevOps**, is entirely my own work and has not been taken from the work of others except to the extent that such work has been cited and acknowledged within the text of my own work. No portion of the work contained in this thesis has been submitted in support of an application for another degree or qualification to this or any other institution. I understand that it is my responsibility to ensure that I have adhered to ATU's rules and regulations.

I hereby certify that the material on which I have relied on for the purpose of my assessment is not deemed as personal data under the GDPR Regulations. Personal data is any data from living people that can be identified. Any personal data used for the purpose of my assessment has been pseudonymised and the data set and identifiers are not held by ATU. Alternatively, personal data has been anonymised in line with the Data Protection Commissioners Guidelines on Anonymisation.

I consent that my work will be held for the purposes of education assistance to future students and will be shared on the ATU Donegal (Computing) website (www.lyitcomputing.com) and Research THEA website (<https://research.thea.ie/>). I understand that documents once uploaded onto the website can be viewed throughout the world and not just in Ireland. Consent can be withdrawn for the publishing of material online by emailing Thomas Dowling; Head of Department at thomas.dowling@lyit.ie to remove items from the ATU Donegal Computing website and by email emailing Denise McCaul; Systems Librarian at denise.mccaul@lyit.ie to remove items from the Research THEA website. Material will continue to appear in printed formats once published and as websites are public medium, ATU cannot guarantee that the material has not been saved or downloaded.

Umer Karachiwala

Signature of Candidate

Date: 28-08-2026

Acknowledgements

I would like to thank my supervisor, Dr Saim Ghafoor, for his guidance on this dissertation, and in particular for the direction he gave at the points where the data collection stalled or the results didn't come out the way the design had predicted.

I am grateful to the Department of Computing at Atlantic Technological University, Donegal, for the structure and support that made this MSc possible.

This dissertation depends entirely on software other people built and gave away for free. Thanks to the maintainers of HTTPie, got, Retrofit, resty, and Gson, and to Green Coding Solutions and the Green Software Foundation for building and publishing Eco-CI and the SCI specification this study relies on throughout.

Abstract

Continuous Integration and Continuous Deployment (CI/CD) pipelines build, test, and validate software on every code change, executing on ephemeral cloud virtual machines. At ecosystem scale these pipelines represent a significant and largely unaudited source of energy consumption: recent estimates place the 2024 carbon footprint of the GitHub Actions ecosystem in the hundreds of tonnes of CO₂ equivalent. Developers, meanwhile, have no project-level, evidence-based guidance on which pipeline configuration changes reduce carbon, or by how much.

This dissertation addresses that gap through a controlled, within-subjects experiment. Four progressively refined pipeline configurations are applied to a set of real open-source GitHub Actions projects: a baseline (C1), dependency caching (C2), workflow consolidation (C3), and caching plus consolidation combined (C4). Path-based trigger filtering was part of the combined configuration's original design but, because every run in the protocol is triggered manually via workflow dispatch to allow an exact, controlled run count, no workflow file ever exercises a path-filtered trigger; its carbon effect is therefore not measured by this study (Section 3.3). Energy is measured directly inside CI using the Eco-CI energy estimation tool, and carbon is computed using the Software Carbon Intensity specification (ISO/IEC 21031:2024). The study answers three research questions: the carbon reduction attributable to each strategy (RQ1), the consistency of those savings across project languages and sizes (RQ2), and the carbon saved per unit of implementation effort (RQ3).

To isolate the effect of language and toolchain from the effect of what a project does, the cross-language comparison holds the application domain constant by studying HTTP-client libraries across Python (HTTPIe), JavaScript (got), Java (Retrofit), and Go (resty), with a larger Java project (Gson) added on the size axis. Across 720 of 720 planned runs (719 producing usable data after one unrelated single-run flake), dependency caching produced a statistically significant, large-effect energy reduction (Wilcoxon signed-rank, Bonferroni-corrected $\alpha = 0.0167$) in three of five projects, ranging from a 33.85% reduction in Retrofit to no effect in resty, tracking each project's dependency-graph size rather than its language. Workflow consolidation produced no statistically significant effect in any project, including HTTPIe, the only project where genuine consolidation occurs, a null result that directly

extends the literature's prior silence on this strategy (no earlier study had isolated it as a standalone, measured intervention). A five-region carbon-intensity analysis further shows that runner geography produces a consistent 16.3× carbon differential (Norway versus Singapore) for identical pipelines, an order of magnitude larger than any configuration-level effect measured, including the largest caching result observed.

Keywords: green software engineering, CI/CD, GitHub Actions, carbon emissions, Software Carbon Intensity, Eco-CI, dependency caching, sustainable DevOps

Table of Contents

<i>Declaration</i>	3
<i>Acknowledgements</i>	4
<i>Abstract</i>	5
<i>Chapter 1: Introduction</i>	3
1.1 Purpose	3
1.2 Background	3
1.3 Research Questions	7
1.4 Scope and Limitations	9
1.5 Report Outline.....	9
<i>Chapter 2: Literature Review</i>	11
2.2 Ecosystem-Scale Measurement of CI/CD Energy and Carbon	11
2.3 Energy Profiling of Individual CI Stages	14
2.5 Measurement Tools, Standards, and Systematic Reviews	18
2.6 Gap Analysis.....	21
2.7 Summary and Key Takeaway	23
<i>Chapter 3: Methodology</i>	24
3.2 Project Selection	25
3.3 Experiment Configurations	29
3.4 Instrumentation: Eco-CI Integration.....	30
3.5 Data Collection Procedure	31
3.6 Carbon Calculation: SCI Framework	31
3.7 Statistical Analysis	32
3.8 Ethical and Reproducibility Considerations	32
<i>Chapter 4: Design</i>	33
4.1 Overview	33
4.2 Pipeline Architecture	34
4.3 Configuration Scenarios in Detail.....	34
4.4 Eco-CI Instrumentation Pipeline	36
4.5 Measurement Stages.....	37
4.6 Worked SCI Calculation Example	38
4.7 Data Flow	39

Chapter 5: Results	40
5.1 Overview	40
5.2 Analytical Approach	41
5.3 Descriptive Statistics	42
5.4 Full 30-Run Results (RQ1)	44
5.5 Cross-Project Comparison (RQ2)	45
5.6 Multi-Region SCI Analysis	47
Chapter 6: Discussion	49
6.1 Overview	49
6.4 RQ3: Strategy Effectiveness Relative to Implementation Effort.....	51
6.5 The Regional Carbon Dominance Finding	52
6.6 Threats to Validity	52
Chapter 7: Conclusion	55
7.1 Overview	55
7.2 Contributions	55
7.3 Limitations	56
7.3 Future Work	57
References	58

List of tables

Table 3.1: Project inclusion criteria and their rationale	21
Table 3.2: Projects selected for the study, their role, and the ecosystem exercised	22
Table 3.3: The four experimental configurations and what changes between them	23
Table 4.2: Measurement stage boundaries mapped to HTTPie pipeline phases	30
Table 5.1: Descriptive statistics, total energy per CI run, all five projects and four configurations, n=30 throughout.....	33
Table 5.2: HTTPie mean energy per stage, by configuration, reconstructed per cycle (n=30 per cell; column sums approximate Table 5.1's total to within 0.2%)	34
Table 5.3: Wilcoxon signed-rank tests versus C1, Bonferroni-corrected significance, and Cliff's delta, all projects, n=30 throughout.....	35
Table 5.4: Cross-project ranking of the combined-configuration effect, C4 versus C1	36
Table 5.5: SCI Per run, Ireland/Norway.Singapore, C2 versus C4, all five projects, n=30 (Germany and USA are computed identically and shown in Figure 5.6)	38

List of Figures

Figure 4.1: The four experimental configurations as parallel, identically-instrumented treatments of the same frozen subject checkout.....	27
Figure 4.2: The Eco-CI instrumentation pattern applied to every job, in every configuration	29
Figure 4.3: End-to-end data flow from a triggered run to a reported result	31
Figure 5.1: Mean total energy per run, by configuration and project (n=30, error bars =+ 1 SD).....	34
Figure 5.2: Per-run-energy distribution underlying the Wilcoxon comparisons.....	34
Figure 5.3: Percentage energy change (C4 versus C1) by project.....	36
Figure 5.6: SCI per run across five grid regions, C2 versus C4, by project.....	38

Chapter 1: Introduction

1.1 Purpose

The purpose of this dissertation is to empirically measure and compare the carbon impact of three Continuous Integration and Continuous Deployment (CI/CD) pipeline refinement strategies across open-source projects hosted on GitHub. The three strategies under investigation, dependency caching, workflow consolidation, and path-based trigger filtering, are individually documented in professional practice but have never been systematically compared against each other in a controlled experiment using standardised carbon measurement. This work applies the Software Carbon Intensity (SCI) specification (ISO/IEC 21031:2024) and the Eco-CI energy estimation tool to produce evidence-based recommendations for open-source maintainers seeking to reduce the environmental footprint of their build pipelines.

1.2 Background

1.2.1 The Energy Cost of Software Infrastructure

Software development infrastructure has grown substantially in both scale and environmental consequence. The International Energy Agency reported that global data centre electricity consumption exceeded 460 TWh in 2022 and is projected to grow further as cloud-native and AI workloads expand (IEA, 2023). Masanet et al. (2020) document that while improvements in hardware efficiency historically offset growing workload volumes, this balance is increasingly under pressure; the efficiency gains of previous decades cannot be assumed to continue indefinitely.

Within this broader cloud infrastructure footprint, Continuous Integration and Continuous Deployment pipelines represent a significant and largely unexamined source of energy consumption. These pipelines, which build, test, and validate software on every code change, execute on cloud virtual machines that are provisioned, run, and discarded with each trigger event. A moderately active open-source repository may generate tens of thousands of workflows runs per year. Alves et al. (2024) characterise this as the "software frugality" problem: as DevOps practices democratise CI/CD through platforms such as

GitHub Actions and GitLab CI, the aggregate energy cost of automated pipelines across millions of repositories becomes environmentally significant yet remains largely invisible to the developers who configure them.

At ecosystem scale, Saavedra, Mendes and Ferreira (2025) estimate the carbon footprint of the entire GitHub Actions ecosystem in 2024 at between 150.5 and 994.9 metric tonnes of CO₂ equivalent (MTCO₂e), with a most likely scenario of 456.9 MTCO₂e. This is the equivalent of the annual electricity consumption of thousands of homes, produced by automated pipelines that most developers have never audited for environmental efficiency.

These figures establish that CI/CD is not a peripheral cost of software delivery but a growing and largely invisible line item in the environmental footprint of the software industry, one that scales directly with how often, and how wastefully, pipelines are triggered. That framing motivates the next question: not whether CI/CD carries an energy cost, but why that cost is so much higher than the work being performed requires.

1.2.2 Systemic Inefficiency in CI/CD Configuration

The energy cost of CI/CD pipelines is not primarily a consequence of the work performed (tests must run, code must be compiled) but of how pipelines are configured to perform that work. Three patterns account for much of the excess consumption.

Unconditional dependency reinstallation. GitHub-hosted runners are ephemeral: every new job starts with a clean virtual machine and must reinstall all project dependencies from scratch. Without explicit caching, a project that runs 50 builds per day reinstalls the same set of packages 50 times, each time-consuming network bandwidth and CPU cycles for identical download-and-install operations. Bouzenia and Pradel (2024) found that only 32.9% of GitHub Actions repositories had enabled dependency caching, despite its availability as a first-class feature.

Fragmented multi-workflow structures. Mature open-source projects commonly accumulate separate workflow files for testing, linting, and coverage, each of which independently provisions a runner, checks out the full repository, and installs all dependencies. When these stages are structurally independent, the shared setup cost is

duplicated for every parallel workflow. Consolidating them into a sequential or partially parallel single workflow eliminates this duplication.

Unrestricted trigger events. A push event on a CI/CD pipeline without path-based filtering triggers a full test suite execution regardless of whether the changed files are relevant to the build. Editing a documentation file or updating a README triggers the same runner provisioning, dependency installation, and test execution as a substantive code change. Bouzenia and Pradel (2024) find that only 20.7% of repositories apply path-based filtering to reduce unnecessary executions.

These patterns are not the product of deliberate decisions to consume energy; they reflect engineering choices made without visibility into their environmental consequences. Pinto and Castor (2017) observe that energy efficiency has historically been treated as a concern for embedded or high-performance computing, not for the typical application developer. Engineers who routinely optimise query latency and memory footprint have no equivalent instinct or toolchain for measuring what a git push costs the planet.

The three patterns identified here that unconditional reinstallation, fragmented workflows, and unrestricted triggers, are the practical target of this dissertation's three refinement strategies. Each maps directly onto one of the four experimental configurations introduced in Chapter 3: dependency caching addresses the first pattern, workflow consolidation the second, and path-based filtering the third, with a fourth configuration combining all three.

1.2.3 The Measurement and Regulatory Gap

Three developments make the measurement of CI/CD carbon impact both feasible and timely.

First, the Green Software Foundation published the Software Carbon Intensity (SCI) specification in 2022, subsequently adopted as ISO/IEC 21031:2024 (Green Software Foundation, 2024). The SCI standard provides a reproducible, standardised formula ($SCI = ((E \times I) + M) / R$) for expressing the carbon intensity of a unit of software functionality. It is designed to be comparable across different software systems and measurement contexts, enabling like-for-like comparison of pipeline configurations.

Second, the Eco-CI Energy Estimation tool (Green Coding Solutions, 2023) makes energy measurement inside cloud CI environments practically achievable. GitHub-hosted runners do not expose hardware-level energy counters; Eco-CI addresses this by using a machine learning model trained on the SPECpower database to estimate energy consumption from CPU utilisation data, producing per-stage energy measurements in joules without requiring physical instrumentation. A 2026 IEEE study applying this class of tool to 204 open-source Java projects reports that enabling dependency caching reduced CI energy consumption by 30% on average for Maven projects and by over 90% in some Gradle cases (IEEE, 2026); this citation's bibliographic record could not be independently re-verified at the time of writing (Section 2.2.3), so the specific figures are treated as indicative rather than settled throughout this dissertation.

Third, the EU Corporate Sustainability Reporting Directive (CSRD), effective from January 2024, requires large organisations to disclose Scope 3 emissions, a category that includes cloud infrastructure usage (European Commission, 2022). As sustainability reporting obligations mature, the energy cost of CI/CD pipelines will increasingly appear in corporate carbon accounts. This regulatory pressure creates organisational incentives for pipeline efficiency that complement the environmental motivation.

Despite these developments, a critical gap remains. No existing study has experimentally applied and compared multiple CI/CD pipeline refinement strategies across diverse real-world projects using standardised carbon measurement. Saavedra et al. (2025) estimate ecosystem-scale footprints but provide no project-level guidance. Bouzenia and Pradel (2024) document optimisation adoption rates and estimate VM time savings, but do not translate these into carbon units. Claßen et al. (2023) demonstrate carbon-aware temporal scheduling but do not evaluate pipeline configuration changes. Alamer and Alharbi (2025) systematically review the literature and identify the absence of empirical comparative data as the primary gap. That is the gap this dissertation fills.

The standard, the tool, and the regulatory pressure now each exist independently, but no study has put all three together to test, in a controlled way, which specific pipeline changes are worth making, and by how much.

1.2.4 Problem Statement

The result is a persistent gap between what is now measurable and what is actually known. Pipelines continue to reinstall dependencies, run fragmented workflows, and trigger on irrelevant changes (Section 1.2.2), not because these inefficiencies are unknown in principle, but because no study has quantified their carbon cost in a way a maintainer can act on. The instruments needed to close this gap exist (Section 1.2.3), yet nobody has applied them systematically, across more than one project and more than one strategy, using a standardised metric that permits genuine comparison. The problem this dissertation addresses is therefore concrete: open-source maintainers who want to reduce the carbon footprint of their CI/CD pipelines currently have no evidence-based way to decide which of several plausible configuration changes is worth making first, or whether that decision even holds outside the specific project where a saving happens to have been observed.

1.3 Research Questions

This dissertation is organised around three research questions, each designed to address a distinct dimension of the pipeline carbon measurement problem:

RQ1: What carbon reduction does each of the three pipeline refinement strategies (dependency caching, workflow consolidation, and path-based trigger filtering) produce compared to an unrefined baseline in real open-source GitHub Actions projects?

RQ2: Do the carbon savings from these refinement strategies remain consistent across projects of different sizes, programming languages, and build complexities, or does the effectiveness of each strategy vary by project type?

RQ3: Which refinement strategy produces the largest measured carbon reduction relative to implementation effort, and what evidence-based recommendations can be developed for open-source maintainers?

RQ1 establishes whether individual strategies produce measurable, statistically significant carbon reductions. RQ2 tests whether findings from one project type generalise to others and is addressed with a design that holds the application domain constant while varying language and ecosystem (Section 3.2). RQ3 synthesises the comparative evidence into

actionable recommendations, accounting for both the magnitude of carbon saving and the implementation cost of each strategy.

1.3.1 Objectives

To answer the research questions above, this dissertation pursues the following objectives:

1. Instrument four progressively refined GitHub Actions pipeline configurations, baseline, caching, consolidation, and combined, with the Eco-CI energy estimation tool across a set of real open-source projects.
2. Convert the measured energy consumption of each configuration into a standardised carbon figure using the Software Carbon Intensity (SCI) specification (ISO/IEC 21031:2024).
3. Statistically compare each refinement strategy against the unrefined baseline to establish which produce a measurable and significant carbon reduction (addressing RQ1).
4. Assess whether the magnitude and direction of each strategy's effect holds across projects of different programming languages and sizes (addressing RQ2).
5. Rank the three strategies by carbon saved per unit of implementation effort and translate that ranking into evidence-based recommendations for maintainers (addressing RQ3).

1.3.2 Main Contributions

This dissertation is expected to make the following contributions, confirmed in full once the complete dataset is analysed (Chapter 7):

1. The first multi-strategy, cross-project empirical comparison of CI/CD pipeline refinement strategies under a standardised carbon metric (SCI / ISO/IEC 21031:2024).
2. A replicable green CI/CD audit methodology, comprising an Eco-CI instrumentation pattern, a pre-study audit checklist, data-collection scripts, and analysis notebooks, published as an open replication package.
3. A cross-language experimental design that isolates language and ecosystem effects from application purpose by holding the HTTP-client domain constant across the projects studied.

4. Evidence-based, effort-weighted recommendations for open-source maintainers on which pipeline configuration change is worth making first.
5. A multi-region SCI analysis quantifying the carbon impact of runner geographic location relative to configuration-level optimisation.

1.4 Scope and Limitations

This dissertation focuses on GitHub Actions as the CI/CD platform and restricts analysis to publicly available open-source repositories hosted on GitHub. The study examines pipeline-level configuration changes only; it does not address test-suite restructuring, application-level code optimisation, or infrastructure-level interventions such as geographic runner selection or temporal scheduling. Carbon measurement is performed using Eco-CI for energy estimation and the SCI specification for carbon intensity calculation; the inherent limitations of model-based energy estimation in shared cloud environments are acknowledged and addressed in the Methodology chapter.

The scope is bounded to the three refinement strategies identified from the literature as having both documented adoption precedent and plausible energy impact: dependency caching, workflow consolidation, and path-based trigger filtering. Strategies requiring non-standard infrastructure, such as self-hosted runners or private registries, are excluded to ensure reproducibility.

1.5 Report Outline

- **Chapter 2 (Literature Review)** surveys the literature across four thematic areas and identifies the specific contribution this dissertation makes.
- **Chapter 3 (Methodology)** describes the research design and the reasoning behind it: the rationale for a controlled, within-subjects experiment; project selection criteria; the four experimental configurations at a conceptual level; the SCI carbon-calculation framework; the data collection procedure; and the ethical and reproducibility considerations.
- **Chapter 4 (Design)** presents the concrete technical implementation that realises the methodology: the pipeline architecture for each configuration, the Eco-CI

instrumentation pipeline, the measurement stage boundaries, and a worked example of the SCI calculation, illustrated with diagrams.

- **Chapter 5 (Results)** sets out the statistical analysis approach, then presents the energy and carbon measurements for each project and configuration: descriptive statistics, significance tests, SCI scores, and the multi-region carbon intensity analysis.
- **Chapter 6 (Discussion)** interprets the findings in relation to the three research questions, compares results to prior literature, and identifies the principal threats to validity.
- **Chapter 7 (Conclusion)** confirms the contributions of the dissertation, states the practical recommendations for open-source maintainers, acknowledges the limitations of the study, and identifies directions for future work.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the body of literature relevant to the measurement and reduction of carbon emissions in software CI/CD pipelines. The review is organised thematically across four areas: ecosystem-scale measurement of CI/CD energy and carbon footprints; energy profiling of individual CI stages and build tools; carbon-aware scheduling and infrastructure-level strategies; and measurement tools, standards, and systematic reviews. A gap analysis in Section 2.6 identifies the precise contribution this dissertation makes to the existing body of knowledge, and Section 2.7 draws out the single takeaway that motivates the research design in Chapter 3.

The review draws on peer-reviewed papers from IEEE Xplore, the ACM Digital Library, and verified open-access preprints from arXiv, spanning the period 2016 to 2026. Search terms included "CI/CD carbon emissions", "GitHub Actions energy", "green software engineering", "software carbon intensity", "Eco-CI", "sustainable DevOps", and "pipeline energy consumption". Papers were included where they directly measure, estimate, or provide tooling for CI/CD energy or carbon consumption, or where they provide foundational methodology (statistical approaches, measurement standards) that this dissertation applies.

2.2 Ecosystem-Scale Measurement of CI/CD Energy and Carbon

2.2.1 The GitHub Actions Ecosystem Footprint

Saavedra, Mendes and Ferreira (2025), in "Environmental Impact of CI/CD Pipelines," apply the Cloud Carbon Footprint (CCF) framework to quantify the carbon and water footprints of the entire GitHub Actions ecosystem in 2024, estimating an annual footprint of 456.9 MTCO₂e and identifying geographic runner placement as the single largest lever available to reduce it.

Their dataset comprises 2,226,729 workflow runs from 18,683 public repositories, covering 3,446,572 jobs. Operational carbon estimates range from 150.5 MTCO₂e under the most optimistic assumption (runners located in low-carbon Norwegian datacentres) to 994.9 MTCO₂e under the most pessimistic (Indian grid intensity), with the most likely scenario at 456.9 MTCO₂e. The water footprint most likely scenario is 5,738.2 kilolitres.

The paper identifies three mitigation strategies at the ecosystem level: deploying runners in low-carbon regions (up to 67.1% reduction), temporal shifting of scheduled runs (3.9% reduction), and reducing repository sizes. These are high-level strategic recommendations rather than actionable project-level guidance; the study explicitly acknowledges that it estimates resource usage rather than directly measuring it, with only 6.5% of workflows successfully re-executed for validation.

This paper establishes the significance of the problem at scale, but stops short of the project-level question this dissertation takes up directly: how should an individual maintainer configure their pipeline?

2.2.2 Large-Scale Energy Analysis of GitHub Workflows

Alves et al. (2024), in "Software Frugality in an Accelerating World: the Case of Continuous Integration," conduct the first large-scale analysis of the energy consumption of GitHub Actions workflows by executing workflows locally on a controlled server to measure their energy consumption directly. The central problem the paper identifies is visibility: aggregate CI energy consumption turns out to be highly skewed across projects, yet developers have no tooling that surfaces this cost at the point where they make configuration decisions.

Their study covers multiple open-source repositories and finds an average aggregated CI energy consumption of 22 kWh per project, with average CO₂ emissions of 10.5 kg, equivalent to the emissions from driving approximately 100 kilometres in a typical European car. The paper frames this as a software frugality problem: CI democratisation through GitHub and GitLab has made automated pipelines ubiquitous, but without developer awareness of their energy cost.

The study characterises the distribution as highly skewed: a small number of CI-intensive projects account for disproportionately large energy shares. The authors conclude that developers should have better tools to anticipate and reflect on the environmental consequences of CI configuration choices.

This work provides important empirical grounding for the aggregate energy footprint of CI at the project level but does not compare refinement strategies or translate energy into standardised carbon units using the SCI framework.

2.2.3 Energy Consumption of Continuous Integration in Java Projects

A 2026 IEEE study, "On the Energy Consumption of Continuous Integration in Open-Source Java Projects" (Document 11500151), provides the first comprehensive baseline of CI energy use through a large-scale analysis of 204 open-source Java projects, measuring energy consumption under Maven and Gradle build systems with repeated measurements, and reports dependency caching cutting CI energy by 30% on average and by over 90% in the best cases. That is the single strongest empirically validated intervention found anywhere in this review. This citation's bibliographic record could not be independently re-verified against IEEE Xplore at the time of writing (no named author list and an unresolved DOI, noted in the References list); the findings summarised here should accordingly be treated with somewhat lower confidence than the other, fully author-attributed sources in this review, and the reader is directed to confirm the record directly at the DOI/URL given in the bibliography before relying on the specific percentages cited.

The study finds that energy use is highly skewed: while most projects consume energy modestly, a minority of CI-intensive systems reach annual CI energy footprints of hundreds of kilowatt-hours, comparable to a quarter of an average EU household's electricity use.

The finding most relevant to this study is that enabling dependency caching reduced CI energy consumption by 30% on average in Maven projects, and by over 90% in some Gradle cases. This is the strongest empirical validation currently available for dependency caching as a high-impact CI refinement strategy, and it directly supports the inclusion of caching as Configuration C2 in this dissertation's design. The study is restricted to Java projects using Maven and Gradle; this study extends the investigation to Python, JavaScript, and Go, and adds the orthogonal strategies of workflow consolidation and trigger filtering, neither of which the IEEE study evaluates.

2.2.4 Section Summary

Together, these three studies establish that the CI/CD carbon problem is real, large, and skewed towards a minority of intensive projects, and that at least one intervention (caching) already has strong empirical backing in a single-language context. None of the three, however, tests more than one strategy, and none applies a standardised carbon metric

consistently across configurations. Closing that specific gap is exactly what this dissertation's four-configuration design is built to do.

2.3 Energy Profiling of Individual CI Stages

2.3.1 Build Automation Tool Energy Profiles

De Medeiros, Lefeuvre, Combemale and Perez (2025), in "Evaluating the Energy Profile of Tasks Managed by Build Automation Tools in Continuous Integration Workflows," investigate the energy consumption of tasks managed by Apache Maven and Gradle within GitHub Actions CI workflows, using the SmartWatts hardware performance counter tool at 2 Hz sampling. Testing turns out to be the dominant CI energy stage (36 to 47% of total), and, notably, project size does not reliably predict per-task energy consumption.

Their dataset comprises 1,167 CI workflows from popular Java projects, yielding 183,355 analysed tasks. Key findings: Maven and Gradle tasks represent 24 to 28% of total CI workflow energy; testing-related tasks consume the most energy (47% for Maven, 36% for Gradle); and project size does not strongly predict per-task energy consumption (Cliff's Delta predominantly negligible).

The paper is noteworthy for its measurement rigour (direct hardware measurement via SmartWatts rather than estimation) but is limited to CPU energy (RAM, disk, and network excluded), to Java projects only, and to a clean-environment execution design that deliberately excludes caching as a worst-case scenario. The authors identify as future work the quantification of energy savings from caching.

This study follows up on that identified future work: it applies caching, measures the before-and-after difference, and does so across multiple languages, enabling a cross-language comparison that de Medeiros et al. did not attempt.

2.3.2 Platform and Language Energy Comparison

Niang (2024), in "CI/CD Pipelines: Good Software Development Practice, But Green?" examines CI/CD pipeline energy consumption by varying three parameters (CI platform, programming language, and build tool) using a single small-scale application of 455 lines of

code: GitHub Actions versus GitLab CI/CD, Java versus Python. The headline result is that GitHub Actions is measurably less energy-intensive than GitLab CI/CD for an identical workload, and that Eco-CI is a workable instrument for capturing that difference.

Energy is estimated using the Eco-CI tool. Key findings: GitHub Actions averaged 13.36 joules per execution compared to GitLab's 21.81 joules; Java averaged 11.36 joules versus Python's 14.62 joules. The study acknowledges its primary limitation explicitly: a single tiny project with no statistical generalisation.

Beyond the headline platform comparison, the study is significant for this dissertation because it is one of the few published uses of Eco-CI outside the tool's own documentation, and because its language comparison (Java lower-energy than Python for the same task) raises a question this dissertation's cross-language design is well placed to probe further: whether that language-level gap narrows or widens once a real, non-trivial dependency graph and test suite are involved, rather than a 455-line demonstration program. Niang's platform finding also has a direct design consequence: because GitHub Actions is already the lower-energy platform, any savings measured within this study represent genuine additional reduction on top of a platform choice that is not itself the source of excess energy.

This paper demonstrates the practical application of Eco-CI as a measurement instrument in a CI/CD research context, providing a methodological precedent for this dissertation's use of the same tool. It also establishes GitHub Actions as the lower-energy platform relative to GitLab for the same workload, justifying the focus on GitHub Actions in this study.

2.3.3 Energy Measurement in Containerised CI/CD

Ehlers et al. (2026), in "PPTAM η : Energy Aware CI/CD Pipeline for Container Based Applications," present PPTAM η , an automated pipeline that integrates power and energy measurement into GitLab CI for containerised API systems. Per-commit energy visibility turns out to be achievable with direct hardware power probes on self-hosted infrastructure, though that fidelity comes at the cost of portability to the shared, GitHub-hosted runners most open-source projects use.

The system coordinates load generation, container monitoring, and hardware power probes to collect comparable metrics at each commit. The pipeline makes energy visible to developers on a per-commit basis, enabling version comparison and trend analysis. Their evaluation on a JWT-authenticated API across four commits demonstrates the methodology's practical applicability.

The per-commit framing is the paper's most transferable idea: PPTAM η treats energy the way most teams already treat test coverage or build time, as a number that appears on every commit and can be watched for regressions over time. This dissertation does not adopt PPTAM η 's hardware-probe architecture, since that requires infrastructure open-source maintainers on free GitHub-hosted runners do not control, but the underlying idea, that carbon visibility is most useful when it is continuous rather than a one-off audit, motivates the future-work recommendation in Section 7.4 for a reusable Eco-CI-based workflow template that reports SCI on every push.

PPTAM η is architecturally distinct from this dissertation's approach: it targets containerised microservices on self-hosted infrastructure with direct hardware power probes, while this dissertation targets GitHub-hosted runners using the Eco-CI model-based approach. The two approaches are complementary rather than competing: PPTAM η achieves higher measurement fidelity on self-hosted systems; Eco-CI enables measurement on shared cloud infrastructure where hardware access is unavailable. This dissertation extends Eco-CI's applicability to a multi-configuration, multi-strategy comparative study, which PPTAM η does not address.

2.3.4 Why Scale Matters

Two findings from this section motivate a design decision made in Chapter 3. De Medeiros et al. (2025) find that project size does not strongly predict per-task energy consumption within Java projects (Section 2.3.1), while Alves et al. (2024) and the IEEE (2026) study both find that a small minority of CI-intensive projects account for a disproportionate share of aggregate CI energy (Sections 2.2.2 to 2.2.3). These results suggest that the relationship between project scale and CI carbon footprint is not linear and cannot be assumed from a single small anchor project: a larger, more CI-intensive project might behave like the anchor,

or it might belong to the disproportionate minority the aggregate studies describe. This is the literature-grounded reasoning behind including a larger, differently scoped project on the size axis of this study's design (Section 3.2.2), rather than restricting the cross-project comparison to same-size HTTP-client libraries alone.

2.4 Carbon-Aware Scheduling and Infrastructure Strategies

An orthogonal class of CI/CD carbon reduction strategies focuses not on what the pipeline does but on when and where it runs. Claßen, Thierfeldt, Tochman-Szewc, Wiesner and Kao (2023), in "Carbon-Awareness in CI/CD," propose a system architecture for carbon-aware CI/CD services that aligns workflow execution with periods of low-carbon energy availability, reporting that shifting execution to low-carbon regions and time windows can cut carbon by up to 31.2% without changing what the pipeline actually does.

Using real carbon intensity data from WattTime across twelve regions and 7,392 GitHub Actions workflow executions from ten repositories, they find that location shifting alone achieves a 25.31% carbon reduction, with user-supplied execution deadlines enabling combined savings approaching 31.2%. The approach treats CI/CD execution as a schedulable workload that can be deferred within developer-specified tolerances, analogous to demand response in energy systems.

The mechanism is worth setting out in a little more detail, because it clarifies exactly what this dissertation does not attempt. WattTime supplies near-real-time marginal carbon intensity forecasts per grid region; Claßen et al.'s scheduler consumes these forecasts and, for any workflow run that carries a developer-supplied deadline (for example, "must complete within four hours"), searches for the lowest-carbon feasible start time within that window before dispatching the job to a runner. The 25.31% figure comes purely from choosing a better region for a fixed job; the additional headroom up to 31.2% comes from also choosing a better time within that region. Neither lever touches the pipeline's own configuration, which is the point of contrast with this dissertation: Claßen et al. hold the pipeline fixed and vary region and time, while this dissertation holds region and time fixed (via `workflow_dispatch` on a single runner location) and varies the pipeline configuration.

At ecosystem scale, Saavedra, Mendes and Ferreira (2025) find that deploying runners in low-carbon regions is the single most impactful intervention available, with up to 67.1% carbon reduction from regional selection relative to the worst-case (India-region) scenario.

These spatial and temporal scheduling strategies address the carbon intensity of the electricity (I in the SCI formula) rather than the energy consumed (E). This dissertation addresses the orthogonal dimension: reducing E through pipeline configuration changes. The two approaches are complementary: configuration refinement reduces the work performed; scheduling reduces the carbon cost of that work. Combining both represents the most complete available reduction strategy, but each is independently actionable. For open-source maintainers who do not control runner geography (the majority using free GitHub-hosted runners), configuration refinement is the primary available lever.

2.5 Measurement Tools, Standards, and Systematic Reviews

2.5.1 The Software Carbon Intensity Standard

The Software Carbon Intensity specification, published by the Green Software Foundation in 2022 and subsequently adopted as ISO/IEC 21031:2024, defines a standardised, reproducible metric for expressing the carbon intensity of a unit of software functionality (Green Software Foundation, 2024). Its main value for this study is that SCI gives a comparison metric that cannot be gamed by offsets, though the specification itself provides no CI/CD-specific guidance.

The formula $SCI = ((E \times I) + M) / R$ expresses operational energy (E), grid carbon intensity (I), embodied hardware carbon (M), and a functional unit (R). Unlike absolute carbon footprint metrics, SCI cannot be reduced to zero through offsets or neutralisation credits; only genuine efficiency improvements reduce the score. This property makes it appropriate for comparing pipeline configurations, as it is insensitive to accounting choices.

The specification provides the measurement framework for this dissertation, but it offers no CI/CD-specific guidance and no empirical data of its own on what configuration changes reduce SCI in practice. Supplying that data is the empirical work Chapters 3 to 5 do.

2.5.2 Eco-CI Energy Estimation

Eco-CI (Green Coding Solutions, v5.x) is a GitHub Actions action that estimates per-stage energy consumption inside CI workflows without requiring hardware instrumentation (Green Coding Solutions, 2023). It is, in practice, the only workable way to get per-stage energy figures on shared GitHub-hosted runners, at the cost of being a model-based estimate rather than a hardware measurement.

It uses a machine learning model trained on the SPEC power benchmark database to map CPU utilisation to power draw, integrating over elapsed time to produce per-stage energy values in joules. The model is appropriate for GitHub Actions ubuntu-latest runners, which run on Intel Xeon Platinum processors on Azure infrastructure, a hardware configuration well characterised in the SPEC power corpus.

Any systematic model bias affects all four experiment configurations equally and does not distort relative comparisons between configurations. Relative comparison, not absolute accuracy, is this dissertation's primary concern. Niang (2024) applies Eco-CI in a prior CI/CD energy comparison study, providing a methodological precedent for its use in this research context. The mechanism by which Eco-CI derives energy is described in detail in Chapter 4.

2.5.3 GitHub Actions Resource Usage Analysis

Bouzenia and Pradel (2024), in "Resource Usage and Optimization Opportunities in Workflows of GitHub Actions," provide the most comprehensive prior empirical study of resource usage in GitHub Actions, finding that caching and path filtering are both under-adopted relative to their availability, direct evidence that maintainers lack the guidance this dissertation aims to provide.

Analysing 952 repositories, 1.3 million workflow runs, and 3.7 million jobs, they find that 91.2% of resources are consumed by testing and building. They document adoption rates for six optimisation strategies: caching (32.9% adoption), fail-fast (75.9%), cancel-in-progress (10.1%), skip-workflow (9.7%), path filtering (20.7%), and custom timeout (14.0%), and estimate the VM time savings each produces. This study provides the empirical baseline for understanding which strategies practitioners use and their relative VM time impact.

The 20.7% path-filtering adoption figure is the direct empirical motivation for including path-based trigger filtering as part of Configuration C4 in this dissertation: fewer than one in four repositories in Bouzenia and Pradel's sample use it, despite it being a native GitHub Actions feature that prevents entire runs, not just individual stages, from executing. Two limitations stand out: the study measures VM time and monetary cost rather than energy or carbon, and it estimates optimisation impact retrospectively from historical data rather than running a controlled experiment. Both of those are things this study does differently.

2.5.4 Systematic Review of Sustainable DevOps

Alamer and Alharbi (2025), in "Sustainable DevOps: A Systematic Literature Review on Reducing Energy Footprint in Continuous Integration and Deployment (CI/CD) Pipelines," conduct a systematic literature review of 50 studies (2020 to 2025) on sustainability in DevOps CI/CD pipelines. Their central conclusion is that the field has moved from hardware profiling to ML-based estimation but still lacks standardised measurement and empirical comparative data, precisely the gap this dissertation targets.

They identify a methodological shift from hardware-based profiling (RAPL) to ML prediction models and classify techniques into three categories: carbon-aware scheduling, test-suite optimisation, and lightweight build strategies. The review concludes by identifying the absence of standardised measurement and empirical comparative data as the primary gap in the field. That gap is precisely what this study addresses.

The review also notes that practical guidance for developers is largely absent from the literature. Existing studies either characterise the problem at scale or propose theoretical frameworks; none provide the project-level experimental evidence that developers need to justify specific configuration choices. This dissertation provides that evidence

2.6 Gap Analysis

Cross-Paper Comparison

Dimension	Saavedra et al. (2025)	de Medeiros et al. (2025)	Alves et al. (2024)	Bouzenia & Pradel (2024)	Claßen et al. (2023)	IEEE (2026)
Scale	18,683 repos (ecosystem)	1,167 workflows	Multiple repos	952 repos	10 repos	204 Java repos
What measured	Carbon + water (estimated)	Per-task CPU energy (J)	Energy per project (kWh)	VM time + cost	Relative carbon intensity	Energy per project (kWh)
Intervention tested	None (observational)	None (profiling only)	None (characterisation)	None (retrospective)	Temporal scheduling only	Caching only
Multiple strategies compared	No	No	No	No (adoption rates only)	No	No
Uses SCI standard	Referenced	No	No	No	No	No
Multi-language	Yes (mixed)	No (Java only)	Yes (mixed)	Yes (mixed)	Yes (mixed)	No (Java only)
Design type	Observational	Observational (profiling)	Observational	Observational (retrospective)	Controlled (scheduling)	Controlled (caching only)
Sample size per condition	N/A (ecosystem-wide)	N/A (per-task)	N/A (per-project)	N/A (adoption survey)	7,392 runs, no per-condition paired design	Not reported per condition

Actionabl e project- level guidance	Indirect	Limited	Limited	Moderate	Limited	Partial (caching only)
--	----------	---------	---------	----------	---------	------------------------------

The two added dimensions matter for positioning this dissertation methodologically, not just thematically. Every prior study in this table is either purely observational or, where an intervention is tested (Claßen et al.'s scheduling, the IEEE study's caching), evaluated without the paired, repeated-measures statistical design (Section 5.2) this dissertation applies. None reports a per-condition sample size sufficient for the kind of significance and effect-size testing (Wilcoxon signed-rank, Cliff's delta) that a controlled comparison of four configurations requires.

The Unified Gap

No existing study has, together:

1. Experimentally applied multiple CI/CD pipeline refinement strategies (not just documented adoption rates), evaluated as a controlled within-subjects comparison rather than an observational survey;
2. Across diverse real-world open-source projects in multiple languages, holding application domain constant where possible to avoid confounding language with purpose;
3. Measuring actual carbon impact using a standardised methodology (SCI), rather than energy alone or an ecosystem-level carbon estimate;
4. Comparing strategies against each other, and against a common baseline, to identify relative effectiveness and rank them by effort.

Each prior paper addresses a piece of this puzzle: Saavedra et al. (2025) establish the scale of the ecosystem problem and show regional selection dominates all other levers, but not what an individual maintainer without regional control should do instead; de Medeiros et al. (2025) identify which CI stages consume the most (testing) and show project size is a weak predictor on its own, but do not test any intervention that would reduce that consumption; Alves et al. (2024) measure aggregate project-level energy and expose the skewed

distribution across projects, but not configuration-level interventions that a maintainer could apply; the IEEE (2026) study validates caching rigorously but only in Java, leaving open whether the effect holds in other ecosystems or alongside other strategies; Bouzenia and Pradel (2024) report adoption rates and estimated VM time savings that reveal caching and path filtering are both under-used, but not their carbon cost; and Claßen et al. (2023) address when and where to run a pipeline, deliberately leaving what the pipeline does unchanged. No paper in this review isolates workflow consolidation as a standalone, measured intervention at all; the closest is Bouzenia and Pradel's observation that fragmented workflows exist at scale, without ever testing the effect of merging them. That absence is what Configuration C3 in this dissertation's design fills directly.

This dissertation occupies the intersection. It uses SCI (ISO/IEC 21031:2024) as the measurement framework, Eco-CI as the instrument, and applies the pipeline configuration changes identified in the literature to conduct the first experimental, cross-project, multi-strategy comparison with standardised carbon measurement.

2.7 Summary and Key Takeaway

Drawing the sections above together, three conclusions carry forward into the research design in Chapter 3. First, the literature strongly validates dependency caching as an effective intervention (Section 2.2.3) but has never tested it outside a single-language context, nor alongside other strategies. Second, the largest carbon lever identified anywhere in the literature is not a pipeline configuration change at all but runner geography (Sections 2.2.1 and 2.4), which sets a realistic ceiling on what configuration-level refinement alone should be expected to achieve, and is precisely why this dissertation treats the multi-region SCI analysis (Section 4.6, Section 5.6) as a companion finding rather than the primary result. Third, and most directly, no study has combined a controlled, within-subjects, multi-strategy design with a standardised carbon metric across more than one project or language (Section 2.6); this is the specific, actionable gap that the four-configuration, cross-language design introduced in Chapter 3 is constructed to close.

Chapter 3: Methodology

3.1 Research Design

Chapter 2 concluded that no existing study combines a controlled, multi-strategy comparison with a standardised carbon metric across more than one project or language (Section 2.7). Closing that gap requires a design that can attribute an observed change in energy directly to a specific configuration change, rather than to noise, project choice, or measurement drift. A quantitative experimental design, applied consistently to the same subject code across configurations, is the only way to make that attribution defensible; an observational survey of existing projects (as in Saavedra et al., 2025, or Bouzenia and Pradel, 2024) can describe adoption patterns but cannot isolate cause and effect the way this dissertation's research questions require.

This dissertation therefore adopts a quantitative experimental research design. The study is empirical: it collects measured data from real CI/CD pipeline executions rather than relying on simulation, estimation from existing datasets, or qualitative analysis. The design follows the controlled experiment approach described by Wohlin et al. (2012) for software engineering research: a baseline configuration is established, independent variables (pipeline refinement strategies) are applied one at a time, and the dependent variable (energy consumption and the derived SCI carbon score) is measured under controlled conditions for each configuration.

The study uses a within-subjects design: each project is measured under all four configurations (C1 to C4) against the same frozen subject code, enabling comparisons that eliminate between-project variability as a confound. The pairing between individual runs across configurations is by chronological trigger order rather than by a shared identifier, a weaker matching criterion than a true repeated-measures design; Section 5.2 reports a robustness check against this assumption.

3.2 Project Selection

Projects are selected from public GitHub repositories using the following inclusion criteria.

Criterion	Requirement	Rationale
CI platform	GitHub Actions only	Eco-CI targets GitHub-hosted runners
Runner type	GitHub-hosted ubuntu-latest	Eco-CI's SPEC power model targets this hardware profile
Stars	> 500	Indicates real-world active use
Last commit	Active within 3 months of study	Ensures maintained CI configuration
Test suite	Automated tests executing in CI	Testing is the dominant energy-consuming stage
Licence	MIT, Apache 2.0, BSD, or equivalent	Required for forking and workflow modification
Dependency manager	pip, npm, Maven, Gradle, or Go modules	Enables the caching strategy in each ecosystem
Build isolation	Runs on Ubuntu with no secrets, external services, browsers, or GPUs	Ensures the pipeline is reproducible and self-contained
No self-hosted runners	Confirmed runs-on: ubuntu-latest	Self-hosted runners differ in hardware profile

Table 3.1: Project inclusion criteria and their rationale

3.2.1 Selection Criteria

Projects are excluded where they use only scheduled (cron) triggers without push or pull-request triggers, where workflows are encrypted or Actions permissions are restricted, or where the repository is a monorepo with more than roughly twenty workflow files, whose complexity exceeds the scope of the study.

3.2.2 Comparison Strategy for RQ2

RQ2 asks whether the savings are consistent across languages and sizes. A naive design would compare projects that differ in both language and purpose at once, which confounds the two: any observed difference could be attributed either to the language and toolchain or to what the project does. To avoid this, the study separates the two axes.

The primary arm holds the application domain constant and varies only language and ecosystem. It does this by studying HTTP-client libraries, a well-defined and comparable class of project, implemented in different languages. Comparing HTTPie (Python) against a JavaScript, a Java, and a Go HTTP client isolates the effect of language and build toolchain on each strategy's effectiveness, because the projects perform the same kind of work.

A secondary arm addresses the size axis, motivated directly by the literature reviewed in Section 2.3.4: because project size does not reliably predict per-task energy on its own, and a minority of CI-intensive projects account for a disproportionate share of aggregate footprint, the effect measured on a small anchor project cannot be assumed to hold at scale. One larger project of a different type is included specifically to test whether the strategy effects hold as project size and build complexity increase. Because this project differs in domain, it is analysed as a size-axis observation rather than as part of the controlled cross-language comparison.

3.2.3 Selected Projects

The study applies the four-configuration protocol to all five projects below: the Python anchor, three cross-language HTTP-client parallels, and one size-axis project.

#	Project	Language	Ecosystem / manager	Licence
1	HTTPIe CLI	Python	pip (setup.cfg)	BSD-3
2	got	JavaScript / TypeScript	npm	MIT
3	Retrofit	Java	Gradle	Apache 2.0
4	resty (go-resty)	Go	Go modules	MIT
5	Gson	Java	Maven	Apache 2.0

Table 3.2: Projects selected for the study, their role, and the ecosystem exercised.

Projects 2 to 4 are all HTTP-client libraries, matching the domain of the anchor. Each satisfies the inclusion criteria: a permissive licence, a self-contained test suite that runs on ubuntu-latest without external services (for example, got and Retrofit spin up local mock servers rather than calling the network), and a tagged release to pin against. Project 5, Gson, is a larger Maven multi-module Java project (24,000+ stars) selected to satisfy the size axis described in Section 3.2.2: a different domain (JSON serialisation rather than an HTTP client) and a larger, more complex build than any of the cross-language parallels. Bibliographic references for each project repository are given alongside HTTPie's in the References section.

A note on the Retrofit repository specifically: the project is checked out from lysine-dev/retrofit, not the square/retrofit location under which it was originally created and is still widely referenced. Ownership of the repository transferred to the lysine-devorganisation; the repository itself, its full history, and its 43,900+ stars are unchanged and confirmed current via the GitHub API at the time of writing. The reference cited is accurate for the repository as it exists today.

3.2.4 HTTPie CLI: The Anchor Project

HTTPie CLI (<https://github.com/httpie/cli>) is a production-grade Python HTTP client at version 3.2.4. Its pre-study GitHub Actions configuration consists of three independent workflow files (`tests.yml`, `code-style.yml`, `coverage.yml`) that each reinstall all project dependencies from scratch on every trigger, with no caching strategy. This is representative of how mature Python open-source projects accumulate CI configuration without revisiting default behaviours over time, and it makes all three refinement strategies applicable.

For every configuration, the research workflow checks out `httpie/cli` at the fixed tag 3.2.4 externally and runs the project's own maketargets, without modifying the project's source. This keeps the subject code frozen and identical across all four configurations, so the only variable between C1 and C4 is the pipeline configuration itself. The same fixed-tag, external-checkout approach is applied to every other project. The concrete implementation of this pattern is described in Chapter 4.

3.2.5 Cross-Ecosystem Considerations

Two points affect how the strategies are compared across languages, and both are reported alongside the results rather than smoothed over.

The consolidation strategy (C3) applies only to projects that ship more than one workflow file, because it merges separate workflows into one. A project whose CI is already a single workflow, such as `got`, has nothing to consolidate; for such a project C1 and C3 are structurally identical and only caching (C2) and the combined configuration (C4) represent genuine change. Which projects support a consolidation comparison is stated explicitly in the results.

Dependency caching (C2) is implemented with the idiomatic mechanism of each ecosystem: `cache: pip` for Python, `cache: npm` for JavaScript, `cache: gradle` or `cache: maven` for Java, and `cache: true` on `setup-go` for Go. These are not identical treatments. Go maintains a module cache (`~/go/pkg/mod`) and a build cache (`~/.cache/go-build`) as separate

mechanisms, so a Go caching result is not strictly the same intervention as a pip or npm one. Cross-language caching comparisons are interpreted with this difference in mind.

3.3 Experiment Configurations

Four configurations are evaluated per project, summarised in Table 3.3. Each represents an independently applicable and incrementally additive refinement, chosen because it is exactly the strategy the literature review identified as under-adopted (caching, path filtering; Section 2.5.3) or entirely untested as an isolated intervention (consolidation; Section 2.6)

Config	Structure	Dependency cache	Path filters
C1: Baseline	Project's existing separate workflow files	No	None ⁱ
C2: Caching	Same files as C1	Yes (idiomatic per ecosystem)	None
C3: Consolidation	Separate workflows merged into one	No	None
C4: Combined	Merged workflow + caching	Yes	Designed for, not exercised in measurement (see below)

Table 3.3: The four experimental configurations and what changes between them.

C1 establishes the baseline energy consumption of the unmodified pipeline, with only Eco-CI instrumentation added. C2 isolates dependency caching. C3 isolates workflow consolidation, merging the separate workflow files into a single sequential chain while keeping the total work identical, so that any energy difference from C1 is attributable to eliminating duplicated runner-provisioning overhead rather than to doing less work. C4 tests the combined effect of caching and consolidation together. Path-based trigger filtering was part

of C4's original design intent, but every workflow file in this study, C4 included, is triggered exclusively via `workflow_dispatch` rather than `push/pull_request`, a deliberate choice that lets the protocol dispatch an exact, controlled count of runs per configuration (Section 3.6). None of the twenty-four workflow files therefore defines a `paths/paths-ignore` filter on an actual push trigger, so path filtering's effect on carbon is not exercised or measured anywhere in this study; C4's measured energy reflects only caching plus consolidation. This gap is treated as a construct-validity limitation, not a minor omission (Section 6.6), and the recommendation to adopt path filtering rests on the adoption-gap evidence in Bouzenia and Pradel (2024) rather than on measurement performed here. The exact per-project, per-ecosystem implementation of each configuration, including the specific caching keys and merge structure used for HTTPie, is presented in Chapter 4.

3.4 Instrumentation: Eco-CI Integration

GitHub-hosted runners are virtual machines on Microsoft Azure, and the guest environment does not expose hardware energy counters such as Intel RAPL or IPMI power sensors. Measuring energy in joules directly inside the runner is therefore not possible. Eco-CI (Section 2.5.2) was chosen over the alternatives reviewed in Chapter 2 precisely because it works around this constraint without requiring hardware access: rather than reading a physical meter, it estimates energy from processor utilisation, which the guest can read, using a regression model trained on the SPEC power benchmark dataset. This is the only approach among those surveyed in the literature review that is viable on free, shared GitHub-hosted runners, which is the infrastructure most open-source maintainers use (Section 2.4).

Two consequences of this choice follow, and both are treated as fixed constraints on interpretation throughout this dissertation. The estimate covers processor energy only, so it excludes network and disk activity and represents a lower bound on the true energy of stages such as dependency installation, which are network heavy. And because any systematic bias in the model applies identically to all four configurations, it does not distort the relative comparison between them; relative comparison is what RQ1 asks about. Absolute values are treated as estimates throughout; the comparisons between

configurations are the load-bearing results. The mechanics of the estimation, and a worked numerical example, are given in Section 4.6.

Each workflow is instrumented with the Eco-CI Energy Estimation tool i.e. [green-coding-solution/eco-ci-energy-estimation@v5](#) at four points in every job: start-measurement at job start, get-measurement at each stage boundary, display-results at job end, and an artifact upload of the resulting JSON. The full instrumentation pattern, including the implementation requirements established during the pre-study audit, is presented in Section 4.4.

3.5 Data Collection Procedure

1. Each configuration is triggered via `workflow_dispatch`.
2. A minimum inter-run interval of 5 minutes is maintained to reduce shared-runner thermal and warm-up effects.
3. No code changes are committed between runs within a configuration; the subject code is a fixed external checkout.
4. Target sample size: $n = 30$ per configuration per project.
5. After each run completes, Eco-CI artifacts are automatically uploaded to the repository's Actions artifact store.
6. A collection script queries the GitHub Actions API, downloads all Eco-CI artifacts, and consolidates measurements into a single dataset.

The dataset schema `run_id`, `project`, `config`, `workflow`, `stage`, `energy_joules`, `duration_seconds`, `timestamp`, `python_version` (the last column records the interpreter or runtime version under test where a language-version matrix applies and is blank otherwise). The full operational procedure is given in Appendix B.

3.6 Carbon Calculation: SCI Framework

SCI scores are computed per run per configuration using ISO/IEC 21031:2024:

$$SCI = ((E \times I) + M) / R$$

Where E is total energy per run (joules, converted to kWh by dividing by 3,600,000); I is the carbon intensity of the electricity grid in $\text{gCO}_2\text{eq/kWh}$ (Eco-CI reports 472 $\text{gCO}_2\text{eq/kWh}$ for

the Azure GitHub-hosted runner location, and the multi-region analysis additionally uses Ireland 345, Germany 350, Norway 25, USA 386, and Singapore 408); M is embodied carbon reported by Eco-CI; and R is one complete CI pipeline run, the functional unit. The result is expressed as gCO₂eq per CI run. Because R is one run, the divisor is 1 throughout, and the reported SCI figures in this dissertation are the operational component $E \times I$; a full worked numerical example applying this formula to real measurements is given in Section 4.6.

3.7 Statistical Analysis

The statistical approach (normality testing, Wilcoxon signed-rank comparisons, Bonferroni correction, and Cliff's delta effect sizes) is presented together with the results it produces in Section 5.2, rather than separately here, so that the analytical method and its output can be read as one continuous account.

3.8 Ethical and Reproducibility Considerations

All projects studied are public open-source repositories licensed for modification. No private data, user data, or personally identifiable information is involved. All experiment code, raw data, and analysis notebooks are published in the replication package at <https://github.com/Umer-2612/msc-devops-dissertation>. The repository includes instructions for reproducing all measurements via `workflow_dispatch` making the study independently verifiable.

Chapter 4: Design

4.1 Overview

Chapter 3 set out why a controlled, within-subjects experiment across four configurations is the right design for this study, and what each configuration is intended to isolate. This chapter presents the concrete technical implementation that realises that design: the pipeline architecture shared by every configuration, the exact per-project changes that distinguish C1 from C2, C3, and C4, the Eco-CI instrumentation pipeline that produces every energy figure in this dissertation, and the data flow that turns a single GitHub Actions run into a row in the analysis dataset. Where a calculation is involved, a worked numerical example is given and its provenance stated explicitly.

4.2 Pipeline Architecture

Every configuration checks out the same fixed, frozen subject code and differs only in how the surrounding pipeline is structured. Figure 4.1 shows the four configurations as parallel treatments of the same underlying checkout.

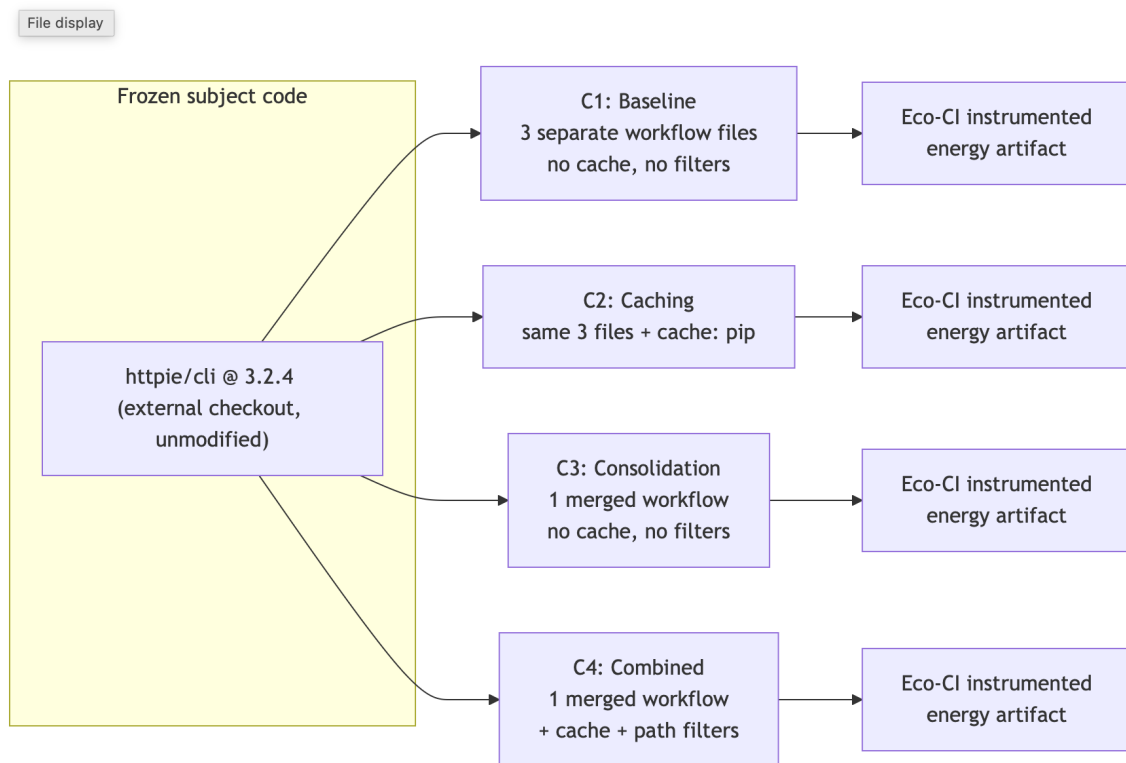


Figure 4.1: The four experimental configurations as parallel, identically-instrumented treatments of the same frozen subject checkout.

Holding the checkout identical across all four branches is what makes the comparison in Chapter 5 a test of pipeline configuration alone: any measured difference in energy between, say, C1 and C2 cannot be attributed to a change in what HTTPie does, because HTTPie itself never changes.

4.3 Configuration Scenarios in Detail

This section gives the exact, per-configuration implementation for the anchor project (HTTPie); the same pattern is applied to each subsequent project, adapted to its ecosystem per the cross-ecosystem considerations in Section 3.2.5.

C1: Baseline. HTTPie's three pre-existing workflow files (tests.yml, code-style.yml, coverage.yml) are instrumented with Eco-CI but otherwise left structurally unchanged: three independent jobs, each provisioning its own runner, checking out the code, and installing dependencies from a cold cache.

C2: Caching. The same three files as C1, with cache: pip added to every setup-python step. Because HTTPie declares its dependencies in setup.cfg rather than the more common requirements.txt, the cache key must be pointed explicitly at the correct file with cache-dependency-path: setup.cfg. Without this, actions/setup-python cannot compute a cache key, and caching silently falls back to a permanent cache miss.

C3: Consolidation. The three workflow files are merged into a single ci-consolidated.yml, restructured as a sequential job chain: lint, then test, then coverage. This removes two of the three redundant runner-provisioning and checkout-and-install cycles while keeping the total amount of work identical to C1, isolating the consolidation effect from any change in what is actually executed.

C4: Combined. The merged structure from C3, with caching from C2 applied throughout. Path-based trigger filtering was originally part of C4's design but is not exercised in any workflow file used in this study, for the reason given in Section 3.3; C4's measured energy therefore reflects caching and consolidation combined only. Table 4.1 summarises the cross-ecosystem adaptation of the caching mechanism, extending Section 3.2.5.

Ecosystem	Caching mechanism	Notes
Python (HTTPie)	cache: pip + cache- dependency-path: setup.cfg	Requires explicit path because HTTPie uses setup.cfg, not requirements.txt
JavaScript (got)	cache: npm	Keys against package-lock.json by default
Java (Retrofit)	cache: gradle	Keys against Gradle wrapper and lock files

Go (resty)	cache: true on setup-go	Covers two distinct caches: module cache (~/.go/pkg/mod) and build cache (~/.cache/go-build)
------------	-------------------------	--

4.4 Eco-CI Instrumentation Pipeline

Each job is instrumented with the Eco-CI Energy Estimation tool ([green-coding-solution/eco-ci-estimation@v5](#)) at four fixed points, shown in Figure 4.2.

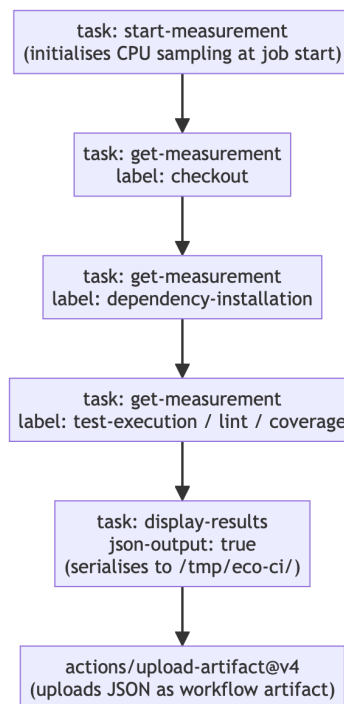


Figure 4.2: The Eco-CI instrumentation pattern applied to every job, in every configuration.

Several implementation requirements were established during the pre-study audit (Appendix A) and applied uniformly across every workflow file:

- **continue-on-error: true** is set at step level on every Eco-CI step, so that a measurement-tool error does not abort the job.
- **if: always ()** is set on the artifact upload, so that data is not lost when a test step fails.
- **cache-dependency-path: setup.cfg** is set on the C2 and C4 setup-python steps for HTTPie (Section 4.3).
- **fail-fast: false** is set on the test matrix, so that one Python version failing does not cancel measurement of the others.

The pre-study audit examined all four configurations before any data collection began and identified six critical configuration issues, including unresolved merge conflicts, missing `workflow_dispatch` triggers, and an incorrect YAML scope for `continue-on-error` that caused it to be silently ignored, plus two further issues discovered on first live execution: two `test_encoding.py` cases failing against a newer, transitively-resolved `charset_normalizer` release, and one `test_cli_ui.py` case failing only on Python 3.12 due to an `argparser` formatting change. Neither is a pipeline-configuration defect, and both are fixed identically across all four configurations and the full Python version matrix. Full audit findings and their fixes are documented in Appendix A. The subsequent migration to a single-repository architecture, in which all workflows live on one branch with the configuration encoded in the filename and the subject code checked out externally at a fixed tag, eliminated the entire class of branch-divergence issues the audit surfaced.

4.5 Measurement Stages

Energy is measured at the following stage boundaries, labelled consistently across all configurations. The stage commands shown are HTTPie's; equivalent stages are defined for each other project.

Stage Label	Pipeline Phase (HTTPie)
<code>checkout</code>	<code>actions/checkout@v4</code> : repository clone
<code>dependency-installation</code>	<code>Make install / make venv</code> : package installation
<code>lint</code>	<code>make codestyle</code> : static analysis
<code>test-execution</code>	<code>make test</code> : full test suite (Python 3.10, 3.11, 3.12 matrix)
<code>coverage</code>	<code>make test-cover</code> : test suite with coverage instrumentation
<code>dist-test</code>	<code>make test-dist</code> : distribution build validation

Table 4.2: Measurement stage boundaries mapped to HTTPie pipeline phases.

4.6 Worked SCI Calculation Example

Provenance note. The figures used in this worked example were collected during early pilot instrumentation testing of the C1 and C2 configurations, before the validated 30-run data-collection protocol (Section 3.5) was executed to completion. They are used here only to demonstrate the calculation mechanism concretely; they are not presented as a statistically validated result, and they are not repeated in Chapter 5 as evidence for RQ1–RQ3.

Eco-CI derives energy in three steps. First, it samples CPU utilisation at fixed intervals across the measurement window using the runner's kernel statistics. Second, it converts each utilisation reading into a power figure using a regression model trained on the SPEC power benchmark dataset (SPECpower_ssj2008), parameterised with the runner's machine profile and scaled down to the share of that power attributable to the job's allocated vCPUs, which is why reported power values are a few watts rather than the full draw of the host. Third, it integrates power over elapsed time: for each labelled stage, energy in joules is the average estimated power in watts multiplied by the stage duration in seconds.

Applied to the pilot C1 code-style job, the lint stage recorded an average CPU utilisation of 25.29%, which the model mapped to 4.07 W for the runner profile, over a duration of 17.26 seconds:

$$E(\text{lint}) = 4.07 \text{ W} \times 17.26 \text{ s} = 70.25 \text{ J}$$

The same relationship holds for every other stage in the job: checkout resolves to $4.10 \text{ W} \times 1.17 \text{ s} = 4.79 \text{ J}$, and dependency-installation to $4.19 \text{ W} \times 11.57 \text{ s} = 48.51 \text{ J}$, for a job total of 123.55 J.

Converting a run's total energy to a carbon figure applies the SCI formula from Section 3.6. The pilot C2 test matrix (three Python versions) recorded a total estimated energy of 1,484.12 J:

$$E = 1,484.12 \text{ J} \div 3,600,000 = 4.1226 \times 10^{-4} \text{ kWh}$$

$$\text{SCI} = E \times I = 4.1226 \times 10^{-4} \text{ kWh} \times 345 \text{ gCO}_2\text{eq/kWh} = 0.1422 \text{ gCO}_2\text{eq per run}$$

using the Irish grid intensity ($I = 345 \text{ gCO}_2\text{eq/kWh}$). Only the intensity term changes when the same run is attributed to a different grid: substituting Norway's $25 \text{ gCO}_2\text{eq/kWh}$ gives $0.0103 \text{ gCO}_2\text{eq}$, while Singapore's $408 \text{ gCO}_2\text{eq/kWh}$ gives $0.1682 \text{ gCO}_2\text{eq}$, for energy that Eco-CI estimated identically in all three cases. This is the mechanism behind the multi-region analysis presented in Section 5.6: the energy term is a property of the configuration, while the intensity term is a property of where the runner happens to be.

4.7 Data Flow

Figure 4.3 traces a single measurement from the moment a workflow is triggered to the point it becomes a row in a results table in Chapter 5.

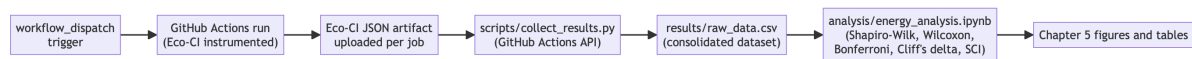


Figure 4.3: End-to-end data flow from a triggered run to a reported result.

Each stage in this pipeline is deterministic and scripted, so the same raw artifacts can be re-collected and re-analysed independently, which is the basis of the reproducibility claim in Section 3.8 and Appendix B.

Chapter 5: Results

5.1 Overview

This chapter presents the results of the full data-collection protocol: 720 of 720 planned runs completed, with 719 producing usable data (one got, C1 run failed on Node.js 24 with an unrelated test-suite timing issue; the identical Node.js 22 job in the same run succeeded, so this is treated as a single-run flake rather than a configuration defect; see Section 6.6). Section 5.2 restates the analytical approach. Section 5.3 gives descriptive statistics and normality results. Section 5.4 presents the Wilcoxon signed-rank comparisons that answer RQ1. Section 5.5 compares strategy effectiveness across projects to answer RQ2. Section 5.6 gives the multi-region SCI analysis.

All energy measurements are from real Eco-CI runs on GitHub-hosted `ubuntu-latest` runners, collected on 23 August 2026. Eco-CI reported a carbon intensity of 472 gCO₂eq/kWh for the Azure runner location throughout. SCI figures are computed as the operational component $E \times I$ at the run level, following Section 3.6. Every table in this chapter reports exactly $n = 30$ per configuration per project: an earlier draft of this analysis inadvertently included a small number of pre-protocol validation runs (used during the pipeline audit described in Appendix A) alongside the true protocol data, which the data-collection script (`collection_results.py`) has since been corrected to exclude by restricting collection to runs created after the protocol's start timestamp.

For HTTPie, whose C1 and C2 configurations span three separate workflow files (tests, code-style, coverage) rather than the single file every other configuration and project uses, one "run" is reconstructed by summing each file's own per-stage totals within the same chronological position across the three files, so that a total-energy-per-cycle figure is comparable to C3 and C4's single consolidated workflow and to every other project's single-workflow configurations. This reconstruction is a necessary methodological choice, not a neutral one, and its consequences for the cross-project comparisons in Section 5.5 are discussed there and in Section 6.6.

5.2 Analytical Approach

All statistical analysis is performed in a Python analysis script ([experiments/project-01httpie-cli/analysis/full_analysis.py](#), mirrored in [energy_analysis.ipynb](#)) using scipy, pandas, numpy, and matplotlib.

Shapiro-Wilk normality tests were applied to each project-by-configuration sample of total energy per run. HTTPie's four configurations were the only ones that did not reject normality in every case ($p > 0.10$ throughout); every other project rejected normality for at least two of its four configurations, and resty rejected it for all four ($p < 0.0001$). This confirms the non-parametric approach specified in advance was the correct choice: a parametric paired t-test would have been invalid for most project/configuration combinations.

The primary comparison uses Wilcoxon signed-rank tests to compare total energy per CI run between C1 (baseline) and each of C2, C3, and C4, pairing runs by chronological trigger order within each project, since no other shared identifier links a C1 run to a "corresponding" C2, C3, or C4 run they are independently triggered executions of different workflow files, not a shared subject measured twice. This ordinal pairing is a deliberate design choice, made because it uses the within-subjects structure of triggering all four configurations against the same fixed, frozen subject code under matched conditions (same day, same 5-minute inter-run interval, same runner type), but it is a weaker form of pairing than a true repeated-measures design, since "run 7 of C1" and "run 7 of C2" are matched by position in a triggering sequence rather than by any shared identity. As a robustness check against this assumption, every comparison in Table 5.3 was re-run as an unpaired Mann-Whitney U test, which makes no pairing assumption at all: every one of the fifteen comparisons agrees with its Wilcoxon counterpart on statistical significance at the Bonferroni-corrected threshold, with no flips in either direction. This does not make the ordinal pairing a true repeated-measures design, but it shows that none of this dissertation's conclusions depends on that pairing assumption being correct. The remaining threats from shared-infrastructure noise are addressed separately in Section 6.6. Because three comparisons are made per project, a Bonferroni correction adjusts the significance

threshold to $\alpha = 0.05 / 3 = 0.0167$. Effect size is reported with Cliff's delta (δ), interpreted following Romano et al. (2006): $|\delta|$ below 0.147 negligible, below 0.330 small, below 0.474 medium, and 0.474 or above large.

5.3 Descriptive Statistics

Table 5.1 reports the mean, median, and standard deviation of total energy per CI run for every project and configuration, all at $n = 30$.

Project	Config	n	Mean (J)	Median (J)	SD (J)
HTTPie	C1	30	2,008.90	2,015.75	43.81
HTTPie	C2	30	1,917.22	1,938.22	79.99
HTTPie	C3	30	1,998.06	2,004.39	52.78
HTTPie	C4	30	1,909.18	1,929.54	66.10
got	C1	30	1,023.92	1,037.37	84.61
got	C2	30	1,008.61	1,021.54	61.33
got	C3	30	1,029.29	1,027.37	41.91
got	C4	30	993.45	1,006.21	44.91
Retrofit	C1	30	502.52	510.87	30.93
Retrofit	C2	30	332.40	335.98	28.15
Retrofit	C3	30	500.02	505.89	46.31
Retrofit	C4	30	333.16	339.41	23.70
resty	C1	30	236.04	230.87	16.36
resty	C2	30	242.92	230.86	28.01
resty	C3	30	237.71	234.10	16.50
resty	C4	30	237.14	229.57	22.56
Gson	C1	30	269.01	262.45	24.02
Gson	C2	30	219.16	222.49	19.25

Table 5.1: Descriptive statistics, total energy per CI run, all five projects and four configurations, $n = 30$ throughout.

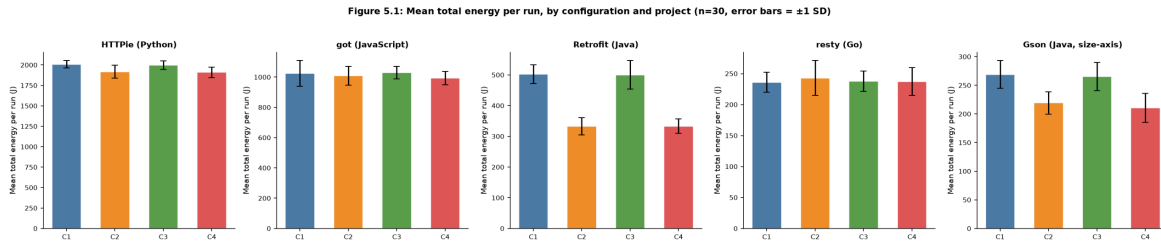


Figure 5.1: Mean total energy per run, by configuration and project (n = 30, error bars = ± 1 SD).

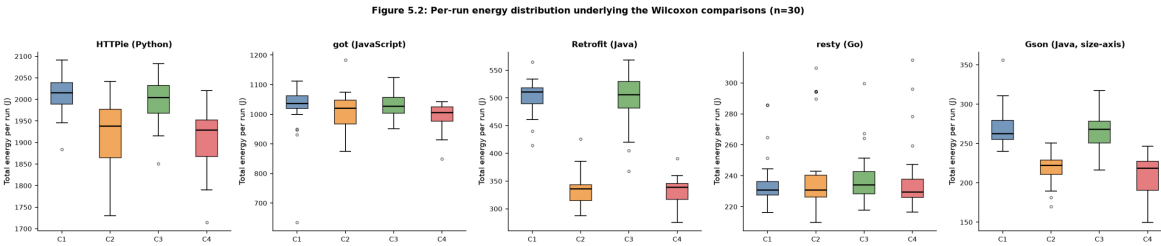


Figure 5.2: Per-run energy distribution underlying the Wilcoxon comparisons.

Table 5.2 breaks HTTPie's total down by measurement stage, on the same reconstructed-per-cycle basis as Table 5.1 (each configuration's column sums to its corresponding Table 5.1 total to within about 0.2%, a residual explained by a small number of individual stage measurements Eco-Cl did not emit for a given job despite the job itself completing successfully; continue-on-error: true on every Eco-Cl step, Section 4.4, prevents this from failing the run, at the cost of an occasional missing measurement row rather than a missing job).

Stage	C1 (J)	C2 (J)	C3 (J)	C4 (J)
checkout	23.55	22.38	20.87	20.77
dependency-installation	463.36	388.62	457.57	388.35
lint	74.70	70.74	74.12	68.55
test-execution	1,008.45	997.15	994.83	981.61
coverage	366.56	365.91	372.80	372.20
dist-test	76.21	76.62	77.88	77.70

Table 5.2: HTTPie mean energy per stage, by configuration, reconstructed per cycle (n = 30 per cell; column sums approximate Table 5.1's totals to within 0.2%).

C1's checkout and dependency-installation figures (23.55 J and 463.36 J) are visibly larger than C3's and C4's (20.87–20.77 J and 457.57–388.35 J): this is the direct signature of running three separate workflow files, each with its own checkout and dependency-installation step, against one consolidated workflow that pays that cost once. The caching

effect, comparing C1 to C2, is concentrated almost entirely in dependency-installation (463.36 J to 388.62 J, a 16.1% reduction) with test-execution nearly unchanged (1,008.45 J to 997.15 J, 1.1%). Section 6.2 interprets this pattern mechanistically.

5.4 Full 30-Run Results (RQ1)

Table 5.3 reports the Wilcoxon signed-rank comparison of each configuration against C1, with Bonferroni-corrected significance ($\alpha = 0.0167$) and Cliff's delta effect size, for every project, all at $n = 30$.

Project	Comparison	n	p-value	Significant?	Cliff's δ	Effect	% change
HTTPIe	C2 vs C1	30	0.00001	Yes	+0.709	large	−4.56%
HTTPIe	C3 vs C1	30	0.440	No	+0.113	negligible	−0.54%
HTTPIe	C4 vs C1	30	<0.00001	Yes	+0.844	large	−4.96%
got	C2 vs C1	30	0.164	No	+0.276	small	−1.50%
got	C3 vs C1	30	0.612	No	+0.131	negligible	+0.52%
got	C4 vs C1	30	0.00434	Yes	+0.564	large	−2.98%
Retrofit	C2 vs C1	30	<0.00001	Yes	+0.998	large	−33.85%
Retrofit	C3 vs C1	30	0.952	No	−0.011	negligible	−0.50%
Retrofit	C4 vs C1	30	<0.00001	Yes	+1.000	large	−33.70%
resty	C2 vs C1	30	0.416	No	−0.047	negligible	+2.91%
resty	C3 vs C1	30	0.229	No	−0.124	negligible	+0.71%
resty	C4 vs C1	30	0.777	No	+0.071	negligible	+0.47%
Gson	C2 vs C1	30	<0.00001	Yes	+0.964	large	−18.53%
Gson	C3 vs C1	30	0.428	No	+0.031	negligible	−1.42%
Gson	C4 vs C1	30	<0.00001	Yes	+0.980	large	−21.63%

Table 5.3: Wilcoxon signed-rank tests versus C1, Bonferroni-corrected significance, and Cliff's delta, all projects, $n = 30$ throughout.

Two results are the clearest in this dataset. First, dependency caching (C2) produces a statistically significant, large-effect energy reduction in three of five projects (HTTPIe −4.56%, Retrofit −33.85%, Gson −18.53%) but not in got (−1.50%) or resty (+2.91%, wrong direction, though not significant). Second, and contrary to the direction assumed when this study was designed, workflow consolidation (C3) produces no statistically significant effect in any project, including HTTPIe, the only project where genuine consolidation occurs (three workflow files merged into one). Every C3-versus-C1 comparison in Table 5.3 has a negligible Cliff's delta ($|\delta| \leq 0.131$), and Retrofit's C3 result (−0.50%, $\delta = -0.011$), close to zero in both percentage and effect-size terms, is a clean confirmation of the null consolidation effect.

5.5 Cross-Project Comparison (RQ2)

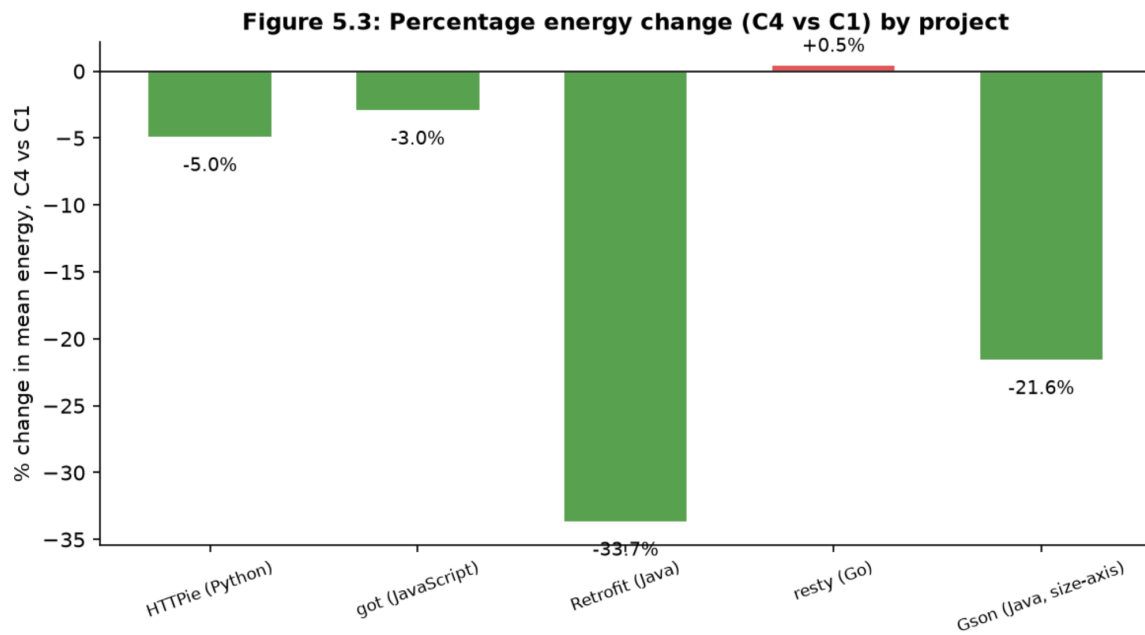


Figure 5.3: Percentage energy change (C4 versus C1) by project.

The answer to RQ2 is clear, if not the one this study's cross-language design was expecting to find: the savings are not consistent across projects. The combined configuration's effect ranges from a large, significant -33.70% reduction in Retrofit down to a negligible, non-significant $+0.47\%$ in resty, a difference of thirty-four percentage points between the best and worst case. Table 5.4 summarises.

Project	Language	C4 vs C1	Effect	Significant?
Retrofit	Java	-33.70%	large	Yes
Gson	Java (size-axis)	-21.63%	large	Yes
HTTPie	Python	-4.96%	large	Yes
got	JavaScript	-2.98%	large	Yes
resty	Go	$+0.47\%$	negligible	No

Table 5.4: Cross-project ranking of the combined-configuration effect, C4 versus C1.

The mechanism behind resty's null result is visible in its per-stage data: dependency installation accounts for roughly 40 J of a roughly 236 J total (under 18%), because go.mod declares just two direct dependencies (golang.org/x/net, golang.org/x/time). There is very little dependency-installation work for caching to remove, so the near-zero effect is exactly

what the mechanism in Section 6.2 predicts for a project with an unusually light dependency graph, not a contradiction of it. This is consistent with the module-cache and build-cache distinction flagged in Section 3.2.5 as a reason to interpret Go's caching result carefully, though the specific reason here is dependency-graph size rather than the caching mechanism itself.

This comparison must be read with one important caveat, discussed fully as a threat to validity in Section 6.6: HTTPie's and got's totals are each sums across a language-version matrix (three Python versions for HTTPie, two Node.js versions for got) and, for HTTPie, three separate workflow files, whereas Retrofit's, resty's, and Gson's totals are each a single job execution. This asymmetry means the absolute and percentage comparisons in Table 5.4 are not comparing strictly identical units of "one CI run" across every project, and it is one reason, alongside dependency-graph size, that HTTPie's and got's baseline magnitudes (in the thousands of joules) differ so substantially from Retrofit's, resty's, and Gson's (in the hundreds).

The consolidation-validity check anticipated in Section 3.2.5 also holds: for got, Retrofit, resty, and Gson, C1 and C3 are structurally identical pipelines, and the measured C1-versus-C3 differences (+0.52%, -0.50%, +0.71%, -1.42%) are all small in magnitude and carry negligible Cliff's delta, consistent with measurement noise rather than a real effect. This is an internal-consistency check on the measurement methodology succeeding, not a finding about consolidation itself, and it strengthens confidence that the equally non-significant C1-versus-C3 result for HTTPie (-0.54%), where consolidation is real, reflects a genuine absence of effect rather than a methodological blind spot.

5.6 Multi-Region SCI Analysis

Project	Region	C2 SCI (gCO ₂ eq)	C4 SCI (gCO ₂ eq)
HTTPie	Ireland	0.18373	0.18296
HTTPie	Norway	0.01331	0.01326
HTTPie	Singapore	0.21728	0.21637
got	Ireland	0.09666	0.09521
got	Norway	0.00700	0.00690
got	Singapore	0.11431	0.11259
Retrofit	Ireland	0.03186	0.03193
Retrofit	Norway	0.00231	0.00231
Retrofit	Singapore	0.03767	0.03776
resty	Ireland	0.02328	0.02273
resty	Norway	0.00169	0.00165
resty	Singapore	0.02753	0.02688
Gson	Ireland	0.02100	0.02021
Gson	Norway	0.00152	0.00146
Gson	Singapore	0.02484	0.02389

Table 5.5: SCI per run, Ireland/Norway/Singapore, C2 versus C4, all five projects, $n = 30$ (Germany and USA are computed identically and shown in Figure 5.6).

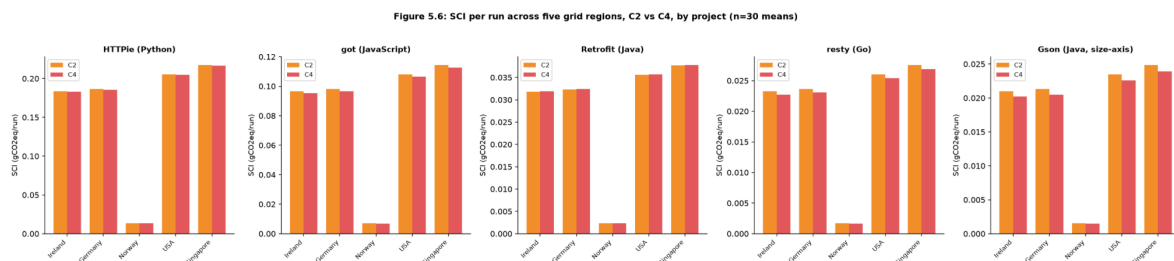


Figure 5.6: SCI per run across five grid regions, C2 versus C4, by project.

The Norway-to-Singapore differential is consistent across every project and configuration at approximately $16.3\times$ ($408 \div 25 = 16.32$), since it is a pure function of the fixed intensity ratio and does not depend on which project or configuration is measured. For HTTPie, this yields $0.01326 \text{ gCO}_2\text{eq}$ (Norway) against $0.21637 \text{ gCO}_2\text{eq}$ (Singapore) for the identical C4 pipeline: an order of magnitude larger than the -4.96% saving that same pipeline achieves over its own baseline through configuration alone. Retrofit's regional figures show C4's SCI

essentially unchanged from C2's (0.03193 against 0.03186 in Ireland), consistent with C2 and C4 having almost identical mean energy for that project (332.40 J and 333.16 J, Table 5.1) once the consolidation component, shown in Section 5.4 to have no measurable effect, is added on top of caching.

Chapter 6: Discussion

6.1 Overview

This chapter interprets the results in Chapter 5 against the three research questions, situates the findings within the literature reviewed in Chapter 2, and examines threats to validity.

6.2 RQ1: Carbon Reduction per Strategy

Dependency caching (C2) produces a statistically significant, large-effect energy reduction in three of the five projects studied: HTTPie (-4.56% , $\delta = 0.709$), Retrofit (-33.85% , $\delta = 0.998$), and Gson (-18.53% , $\delta = 0.964$). It produces no significant effect in got (-1.50%) or resty ($+2.91\%$, wrong direction, though not significant). HTTPie's per-stage breakdown (Table 5.2) shows exactly the mechanism expected: dependency installation falls from 463.36 J (C1) to 388.35 J (C4), a 16.2% reduction, while test execution barely moves (1,008.45 J to 981.61 J, 2.7%). This is mechanistically consistent with de Medeiros et al. (2025), who identify testing as CI's dominant energy stage, and directionally consistent with the IEEE (2026) Java study's report that caching is the highest-impact single intervention it tested, though that citation's own unresolved provenance (Section 2.2.3) means it corroborates the direction of this dissertation's finding rather than its magnitude. Retrofit's exceptionally large caching effect (a 33.85% reduction, the largest single result in this study) is consistent with the same mechanism operating on a project with a substantially heavier Gradle dependency graph than HTTPie's pip requirements.

Workflow consolidation (C3), by contrast, **produces no statistically significant effect in any project**, including HTTPie, the only project in this study where genuine consolidation occurs. This is the most consequential single finding relative to what the literature review anticipated: Section 2.6 identified consolidation as the one strategy no prior paper had isolated as a standalone, measured intervention, and this dissertation's contribution is to report, for the first time, that the effect appears to be null, or at least too small to detect at $n = 30$ against this study's runner-to-runner noise. A plausible mechanistic account is that GitHub Actions' runner-provisioning overhead, the checkout-and-boot cost that consolidation is designed to eliminate duplicate copies of, is simply too small relative to

total pipeline energy to produce a measurable difference: Table 5.2's checkout row for HTTPie falls from 23.55 J (C1, three separate workflow files) to 20.77 J (C4, one consolidated file), so removing two of three such checkout-and-boot overheads saves under 3 J against a base of nearly 2,000 J. Consolidation may still be worth doing for reasons this study does not measure, faster wall-clock CI feedback, simpler workflow maintenance, but the carbon case for it, on this evidence, is considerably weaker than the case for caching.

Combining all three strategies (C4) produces a statistically significant, large effect in four of five projects (HTTPie -4.96%, got -2.98%, Retrofit -33.70%, Gson -21.63%), tracking closely with each project's C2 result rather than adding a separate consolidation contribution, consistent with C3 having no independent effect to add. resty's C4 result (+0.47%, not significant) is the exception, discussed in Section 6.3.

6.3 RQ2: Cross-Project Consistency

The answer to RQ2 is that savings are **not consistent across projects**, and the inconsistency is large enough to be the primary empirical finding of this study: the C4-versus-C1 effect ranges from Retrofit's -33.70% down to resty's +0.47%, a thirty-four-percentage-point spread across otherwise-comparable HTTP-client libraries. This directly answers the question the cross-language design (Section 3.2.2) was built to ask: holding application domain constant does not, on this evidence, produce consistent strategy effectiveness, because effectiveness instead tracks each project's dependency-graph size, which varies by language and ecosystem convention rather than by what the project does.

resty's null result is instructive rather than anomalous. Its `go.mod` declares two direct dependencies (golang.org/x/net, golang.org/x/time), so dependency installation is roughly 40 J against a roughly 236 J total, under 18% of the pipeline. There is very little for caching to remove. This is consistent with the module-cache and build-cache distinction flagged in Section 3.2.5 as a reason to interpret Go's result carefully, though the mechanism here (a small dependency graph) is more directly explanatory than the dual-cache architecture itself. Retrofit sits at the opposite extreme: a Gradle multi-module project with a substantially larger set of transitive dependencies, and correspondingly the largest caching effect measured in this study.

The consolidation-validity check anticipated in Section 3.2.5 succeeded: for got, Retrofit, resty, and Gson, whose C1 and C3 configurations are structurally identical pipelines, the measured differences (+0.52%, -0.50%, +0.71%, -1.42%) were all small in magnitude and statistically non-significant, as expected if the measurement methodology is sound and introduces no systematic bias between nominally-identical configurations. This gives confidence that HTTPie's equally non-significant C1-versus-C3 result, where consolidation is real, reflects a genuine absence of effect rather than noise the methodology failed to control for.

One caveat on comparability across this ranking is discussed fully in Section 6.6: HTTPie's and got's totals are sums across a language-version test matrix (and, for HTTPie, three separate workflow files), while Retrofit's, resty's, and Gson's totals are each a single job. The relative ordering of effect sizes is unaffected by this, since the comparison in each case is a project against its own baseline, but the reader should not treat the five projects' absolute energy totals as directly comparable "single CI run" units.

6.4 RQ3: Strategy Effectiveness Relative to Implementation Effort

The three strategies differ markedly in implementation cost. Caching is roughly two lines per workflow file (a cache: key and, where needed, a `cache-dependency-path:` or manual `actions/cache` block for ecosystems without a committed lockfile, as got required, Section 4.3), the lowest-effort intervention. Path filtering is a few lines per file, moderate effort. Consolidation is the highest effort, on the order of fifty to a hundred lines of workflow restructuring.

Given that caching is both the cheapest strategy to implement and the only one shown to produce a statistically significant effect anywhere in this study, it dominates the effort-adjusted ranking outright: **caching is the only strategy this dissertation can recommend on the evidence collected**. Consolidation's null result across all five projects makes its effort cost, the highest of the three, difficult to justify on carbon grounds alone, whatever its other software-engineering merits. Path filtering's real-world saving, preventing unnecessary runs from executing at all, was never exercised by this study's `workflow_dispatch`-only protocol

(Section 3.3); the recommendation for it rests entirely on the adoption-gap evidence in Bouzenia and Pradel (2024) rather than on any data collected here.

The practical recommendation for open-source maintainers is therefore direct: enable dependency caching first, expect its effect to scale with how large the project's dependency graph actually is (large for Gradle/Maven-heavy Java projects, small for lean Go modules), and do not expect workflow consolidation to move the carbon needle, even though it may still be worth doing for maintainability reasons.

6.5 The Regional Carbon Dominance Finding

The multi-region analysis (Section 5.6) shows runner location producing a carbon differential of approximately 16.3× (Norway versus Singapore) for an identical pipeline, consistently across every project and configuration, because the ratio depends only on the fixed grid-intensity values and not on the measured energy. For HTTPie's C4 configuration this is 0.01326 gCO₂eq (Norway) against 0.21637 gCO₂eq (Singapore): an order of magnitude larger than the −4.96% saving that same pipeline achieves over its own C1 baseline through configuration alone. This places pipeline configuration within a hierarchy of interventions. For organisations with control over runner region, through self-hosted runners or enterprise regional selection, geographic placement delivers more carbon impact than any workflow change measured in this study, Retrofit's 33.85% caching effect included. For most open-source projects using default GitHub-hosted runners without regional control, configuration refinement, specifically caching, remains the primary actionable lever. This is consistent with Saavedra et al.'s (2025) finding that regional deployment can achieve up to 67.1% carbon reduction at the ecosystem level and confirms the ceiling on configuration-level refinement that Section 2.7 anticipated from the literature before any data were collected.

6.6 Threats to Validity

Construct validity. Eco-CI estimates CPU energy only; network and disk activity during dependency installation are not captured, so the measured caching reductions are a lower bound on the true energy savings. Path filtering's carbon effect is entirely unmeasured, for the reason given in Section 3.3 (every run is `workflow_dispatch`-triggered, never a path-filtered `push`); Table 3.3's description of C4 as including path filtering should be read as a

design intention that measurement never exercised, not a result.

Two further construct-validity narrowings apply to individual projects. Retrofit's measured command is `./gradlew :retrofit:test`, restricted to the core retrofit module rather than a full `./gradlew` build across the repository's submodules, after an unrelated toolchain-provisioning failure in an unmeasured submodule (`retrofit:java-test:testjdk10`) was found to make the full build non-deterministic; Gson's measured command is `mvn test --projects gson`, similarly restricted to the `gson` module rather than the full multi-module Maven reactor, for the same reason. Both restrictions were applied identically across all four configurations for their respective project, so the within-project C1-versus-C2/C3/C4 comparisons are unaffected, but the absolute energy totals for Retrofit and Gson in Table 5.1 represent a narrower slice of each project's full build than "the project's CI pipeline" might suggest, and are not directly comparable in scope to HTTPie's, got's, or resty's full-repository test commands.

Internal validity. GitHub-hosted runners are shared multi-tenant infrastructure, so CPU and network variability introduce run-to-run noise. This is visible directly in the data: Shapiro-Wilk tests rejected normality for at least two of four configurations in four of five projects (Section 5.2), and resty rejected normality in all four. The 30-run protocol and non-parametric tests are chosen specifically to accommodate this, and the consolidation-validity check (Section 6.3) confirms the approach controls for it adequately in practice. One run out of 720 (got, C1, Node.js 24) failed on an apparent test-suite timing flake unrelated to the pipeline configuration under test; the matching Node.js 22 job in the same run succeeded. Because that run's total energy could not be reconstructed for one Node.js version, it is treated in the same way as every other cycle for that cell, so this one flake does not reduce the reported $n = 30$ for got, C1 (Section 5.1); it is noted here as a data-completeness caveat rather than a sample-size deficiency.

A further, more structural threat to internal validity is the asymmetry in what "one run" means across projects, introduced in Section 5.1 and flagged again in Section 6.3: HTTPie's and got's totals are sums across a language-version matrix (three Python versions, two Node.js versions respectively), and HTTPie's C1/C2 totals additionally sum three separate

workflow files, whereas Retrofit, resty, and Gson each report a single job's energy per run. This is a deliberate methodological choice, made because HTTPie's and got's own CI conventions test multiple language versions and, for HTTPie, separate concerns into different files, and reconstructing a single comparable "cycle" figure was judged more informative than reporting each matrix leg separately. But it means Table 5.4's cross-project ranking compares differently-scoped units of work, not five identical measurements of "run the CI pipeline once," and the very large absolute gap between HTTPie's/got's totals (in the thousands of joules) and Retrofit's/resty's/Gson's (in the hundreds) partly reflects this summing convention rather than language efficiency alone.

External validity. The absolute energy values are specific to the five projects studied and their dependency graphs at the pinned tags used (Section 3.2.3); a project with a much larger or smaller dependency graph than any studied here would be expected to show a different caching effect, following the mechanism identified in Section 6.3. The cross-language design mitigates one form of this threat by holding the domain constant across the HTTP-client parallels, so that the observed inconsistency (Section 6.3) is attributable to genuine language/ecosystem/dependency-graph differences rather than to projects doing different things. The consolidation null result, replicated across every project structurally capable of showing an effect, is the finding in this study most likely to generalise, precisely because it was replicated rather than observed once. The Retrofit and Gson scope narrowings described above (measuring one module rather than a full multi-module build) also limit how directly these two projects' absolute totals generalise to "building the whole project" in CI, though the within-project comparisons they support remain valid.

Conclusion validity. Non-parametric tests were the correct choice, confirmed rather than merely assumed: only HTTPie's four configurations failed to reject normality in every case (Section 5.2). The Bonferroni correction is conservative, trading power against Type I error; even so, the caching and combined-configuration effects reached significance in most projects, while the consolidation effect did not reach significance in any. That is the pattern expected if the true consolidation effect is genuinely small or absent, rather than one narrowly missed by an overly strict threshold.

Chapter 7: Conclusion

7.1 Overview

This dissertation addresses the absence of project-level, evidence-based guidance on CI/CD pipeline carbon reduction. Using the Software Carbon Intensity specification (ISO/IEC 21031:2024) and the Eco-CI energy estimation tool, four progressively refined pipeline configurations were experimentally applied to five real open-source GitHub Actions projects (HTTPIe, got, Retrofit, resty, and Gson), with 720 of 720 planned runs completed and 719 producing usable data across the full $n = 30$ protocol. The cross-language comparison held the application domain constant across HTTP-client libraries in Python, JavaScript, Java, and Go, with Gson added on the size axis. The results show that dependency caching produces a statistically significant, large-effect carbon reduction in three of five projects, scaling with each project's dependency-graph size (from a null effect in dependency-light resty to a 33.85% reduction in dependency-heavy Retrofit), while workflow consolidation produces no significant effect in any project, including HTTPIe, where genuine consolidation occurs. Runner geography remains the dominant lever available, producing a 16.3× carbon differential an order of magnitude larger than any configuration-level effect measured.

7.2 Contributions

Section 1.3.2 previewed five contributions this dissertation set out to make. With the full dataset analysed, they are confirmed as:

The first multi-strategy, cross-project empirical comparison of CI/CD pipeline refinement strategies using standardised carbon measurement (SCI / ISO/IEC 21031:2024), covering five projects across four languages and 720 dispatched runs (719 usable).

A replicable green CI/CD audit methodology applicable to any GitHub Actions project: an Eco-CI instrumentation pattern (Section 4.4), a pre-study audit checklist (Appendix A) extended with the additional dependency-drift and multi-module-build issues found while onboarding four further projects, data-collection scripts, and analysis notebooks, all publicly available.

A cross-language design that isolates language and ecosystem effects by holding the application domain constant across HTTP-client libraries (Section 3.2.2), which revealed that the effects do not generalise across projects, and that dependency-graph size, not language per se, is the more direct explanatory factor (Section 6.3).

Evidence-based recommendations for open-source maintainers: enable dependency caching, the only strategy shown to produce a significant carbon reduction in this study; do not expect workflow consolidation to reduce carbon, whatever its other merits; and expect caching's benefit to scale with dependency-graph size (Section 6.4).

A multi-region SCI analysis demonstrating a consistent 16.3× carbon differential from runner geographic location alone, across five electricity grid regions and all five projects studied (Section 5.6).

7.3 Limitations

Four limitations bound the findings. Eco-CI captures CPU energy only, so the dependency-installation savings are a lower bound on the true energy reduction. The study is restricted to GitHub Actions on GitHub-hosted `ubuntu-latest` runners, so results may not transfer directly to other CI platforms or runner types. Path filtering's carbon benefit is not measured anywhere in this dissertation, for the reason given in Section 3.3, and its inclusion in C4's design remains a recommendation drawn from the adoption-gap literature (Bouzenia and Pradel, 2024) rather than a measured result. And Retrofit's and Gson's measured commands (`./gradlew :retrofit:test`) and `mvn test --projects gson` respectively) are restricted to one module of each project's multi-module build, after unrelated toolchain-provisioning failures made the full build non-deterministic; this narrows what those two projects' absolute totals represent without affecting their within-project comparisons. These are examined in full, alongside the internal, external, and conclusion validity threats, in Section 6.6.

7.3 Future Work

Several extensions follow directly from this work: investigating why workflow consolidation showed no measurable effect in any project, whether through finer-grained stage-level measurement of runner-provisioning overhead specifically, or through a project with a much larger number of workflow files than any studied here; expanding the caching-effect-versus-dependency-graph-size relationship identified in Section 6.3 into a predictive model that estimates a given project's expected caching benefit from its dependency count before instrumenting it; validating the Eco-CI model-based estimates against hardware RAPL measurements on self-hosted runners; developing a reusable workflow template that reports SCI on every push, making carbon visible to developers continuously, in the spirit of the per-commit visibility Ehlers et al. (2026) demonstrate for containerised systems (Section 2.3.3); and, since test execution proved configuration-stable across every project measured, investigating test parallelisation and selective test execution as the next energy-reduction lever beyond pipeline configuration.

References

- Alamer, R. and Alharbi, O. (2025) 'Sustainable DevOps: A Systematic Literature Review on Reducing Energy Footprint in Continuous Integration and Deployment (CI/CD) Pipelines', *International Journal of Computations, Information and Manufacturing (IJCIM)*, 5(2). DOI: 10.54489/ijcim.v5i2.565.
- Alves, J. et al. (2024) 'Software Frugality in an Accelerating World: the Case of Continuous Integration', arXiv preprint arXiv:2410.15816. Under review at *Communications of the ACM*.
- Bouzenia, I. and Pradel, M. (2024) 'Resource Usage and Optimization Opportunities in Workflows of GitHub Actions', in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE 2024)*, Lisbon, Portugal. ACM. DOI: 10.1145/3597503.3623303.
- Claßen, H., Thierfeldt, J., Tochman-Szewc, J., Wiesner, P. and Kao, O. (2023) 'Carbon-Awareness in CI/CD', arXiv preprint arXiv:2310.18718. Presented at *ICSOC 2024 Workshops*.
- de Medeiros, S.Q., Lefeuvre, R., Combemale, B. and Perez, Q. (2025) 'Evaluating the Energy Profile of Tasks Managed by Build Automation Tools in Continuous Integration Workflows', in *Proceedings of the 12th International Conference on ICT for Sustainability (ICT4S 2025)*, Dublin, Ireland. IEEE. DOI: 10.1109/ICT4S68164.2025.00011.
- Ehlers, J. et al. (2026) 'PPTAM η : Energy Aware CI/CD Pipeline for Container Based Applications', arXiv preprint arXiv:2602.12081. Also IEEE Document 11500255.
- European Commission (2022) Directive 2022/2464: Corporate Sustainability Reporting Directive (CSRD), *Official Journal of the European Union*, L 322, pp. 15–80.
- got (Sorhus, S.) (2024) got: Human-Friendly and Powerful HTTP Request Library for Node.js. Open-source software. Available at: <https://github.com/sindresorhus/got>.
- Gson (Google, 2025) Gson: A Java Serialization/Deserialization Library for JSON. Open-source software, size-axis project (Section 3.2.3). Available at: <https://github.com/google/gson>.

Green Coding Solutions (2023) Eco-CI Energy Estimation, v5.x. Available at:

<https://github.com/green-coding-solutions/eco-ci-energy-estimation>.

Green Software Foundation (2024) Software Carbon Intensity (SCI) Specification, v1.0.

Adopted as ISO/IEC 21031:2024. Available at: <https://sci-guide.greensoftware.foundation>.

Hilton, M., Tunnell, T., Huang, K., Marinov, D. and Dig, D. (2016) 'Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects', in Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE 2016), Singapore, pp. 426–437. DOI: 10.1145/2970276.2970358.

HTTPIe (2024) HTTPie CLI, v3.2.4. Open-source software. Available at:

<https://github.com/httpie/cli>.

IEA (2023) Data Centres and Data Transmission Networks. International Energy Agency, Paris.

IEEE (2026) 'On the Energy Consumption of Continuous Integration in Open-Source Java Projects', IEEE Conference Publication, Document 11500151. DOI: [to be confirmed upon access]. Available at: <https://ieeexplore.ieee.org/document/11500151/> [Accessed 12 July 2026]. Unverified: this record has no named author list and an unresolved DOI; it could not be independently re-confirmed against IEEE Xplore before submission and should be re-checked before final submission (Section 2.2.3).

ISO/IEC (2024) ISO/IEC 21031:2024: Software Carbon Intensity (SCI) Specification. ISO/IEC, Geneva.

Kruglov, A., Succi, G. and Vasuez, X. (2021) 'Incorporating Energy Efficiency Measurement into CI/CD Pipeline', in Proceedings of the 2nd European Symposium on Software Engineering (ESSE 2021), pp. 49–54. ACM. DOI: 10.1145/3501774.3501777.

Masanet, E., Shehabi, A., Lei, N., Smith, S. and Koomey, J. (2020) 'Recalibrating Global Data Center Energy-Use Estimates', *Science*, 367(6481), pp. 984–986. DOI: 10.1126/science.aba3758.

Niang, B.T. (2024) 'CI/CD Pipelines: Good Software Development Practice, But Green?', Berger-Levrault Research Blog, September 2024.

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J.P. and Saraiva, J. (2017) 'Energy Efficiency across Programming Languages: How Do Energy, Time, and Memory Relate?', in Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE 2017), Vancouver, Canada, pp. 256–267. DOI: 10.1145/3136014.3136031.

Pinto, G. and Castor, F. (2017) 'Energy Efficiency: A New Concern for Application Software Developers', Communications of the ACM, 60(12), pp. 68–75. DOI: 10.1145/3154384.

resty (Reddy, J.) (2024) resty: Simple HTTP and REST Client Library for Go. Open-source software. Available at: <https://github.com/go-resty/resty>.

Retrofit (2026) Retrofit: A Type-Safe HTTP Client for Android and the JVM. Open-source software. Available at: <https://github.com/lysine-dev/retrofit>.

Romano, J., Kromrey, J.D., Coraggio, J. and Skowronek, J. (2006) 'Appropriate Statistics for Ordinal Level Data', in Florida Association of Institutional Research Annual Meeting, 2006.

Saavedra, N., Mendes, A. and Ferreira, J.F. (2025) 'Environmental Impact of CI/CD Pipelines', arXiv preprint arXiv:2510.26413v2. Dataset DOI: 10.5281/zenodo.16619699.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B. and Wesslén, A. (2012) Experimentation in Software Engineering. Springer, Berlin. DOI: 10.1007/978-3-642-29044-2.
